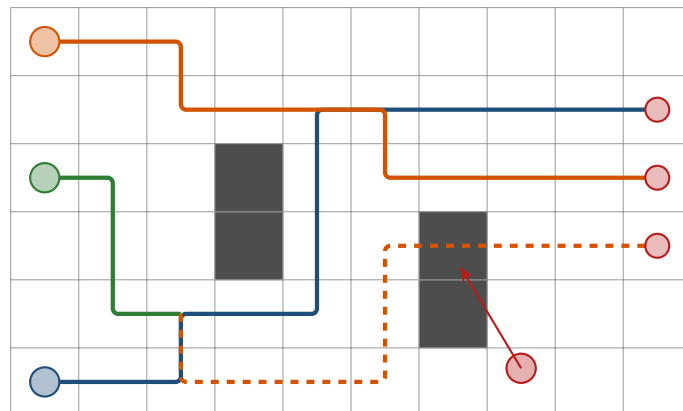


Multi-Agent Path Planning and Drone Collision Avoidance

A Textbook of Algorithms:
From Graph Search to Hybrid Swarm Systems



The Searchbook Project

September 9, 2026

Multi-Agent Path Planning and Drone Collision Avoidance

A Textbook of Algorithms: From Graph Search to Hybrid Swarm Systems

Typeset with L^AT_EX (pdfT_EX), TikZ and pgfplots. Figures were drawn in TikZ or generated from the Python reference implementations that accompany the book.

This book was written to accompany a twelve-week research training plan on multi-agent path planning and collision avoidance for drone swarms. It is intended for self-study. All algorithms described here are established published methods; the original sources are cited in each chapter and collected in the bibliography.

Source code, figures and L^AT_EX sources are distributed together with this book so that every example can be reproduced and every figure regenerated.

Compiled on September 9, 2026.

Contents

Preface	v
Notation	viii
I Foundations	1
II Single-Agent Graph Search	2
III Multi-Agent Path Finding	3
IV Local and Reactive Collision Avoidance	4
1 Artificial Potential Fields	5
1.1 Why this matters for a drone swarm	5
1.2 The idea in plain words	6
1.3 Problem statement and notation	7
1.4 The algorithm	8
1.5 A worked example	10
1.6 Properties: continuity, descent, safety, and where the minimum is	11
1.7 The failure modes	13
1.8 Remedies	18
1.9 Swarms: inter-agent repulsion, flocking and consensus	20
1.10 Implementation notes	21
1.11 Where this fits in the drone system	23
1.12 Summary	23
1.13 Exercises	24

V	Sampling-Based Motion Planning	26
VI	Tracking and Prediction	27
VII	Optimization and Control	28
VIII	Putting It Together	29
IX	Appendices	30
	Bibliography	31
	Index	32

List of Algorithms

1.1	The potential-field force at a point: hybrid attraction, Khatib repulsion, optional GNRON factor	9
1.2	Gradient-descent controller with a velocity limit and local-minimum detection .	10

Preface

Why this book exists

Imagine a small fleet of drones that must fly through the same airspace at the same time. Each drone has a job to do and a route to fly. Before take-off, a planner must make sure that no two routes collide. During the flight, something unplanned happens: another drone, not part of the fleet and not talking to it, crosses the formation. The fleet must notice it, guess where it is going, get out of its way, and then return to the plan—or, if that is impossible, make a new plan and check it against the rest of the fleet.

Every part of that story is a well-studied algorithmic problem. Planning a route is *graph search*. Planning routes for many drones at once without collisions is *multi-agent path finding*. Reacting to a surprise in a fraction of a second is *local collision avoidance*. Guessing where the intruder is going is *tracking and trajectory prediction*. Making a new plan without starting from scratch is *incremental replanning*. Keeping the fleet in formation and within radio range while doing all of this is *control*. The individual algorithms are established and published. What is not settled is how to combine them into one system that keeps the guarantees of the global plan, reacts fast enough to be safe, and can be evaluated honestly.

This book teaches all of those algorithms, one at a time, from first principles, and then shows how they fit together into a hybrid swarm planner. It was written to accompany a twelve-week research training plan; the plan is reproduced, with week-by-week reading and coding assignments, in ??.

Who should read it

The book is for a graduate student or engineer who is starting research on drone swarms, or on multi-robot motion planning more generally, and who wants to *implement* the algorithms rather than merely recognise their names. You should be comfortable with Python and with the mathematics of a first course in linear algebra, calculus and probability. Everything else is introduced when it is needed, and ?? collects the mathematical facts that the book relies on. No prior knowledge of robotics or planning is assumed.

What you will be able to do

When you have worked through the book and its exercises you will be able to

- implement and explain A^* and space-time A^* , and say exactly why they return optimal paths under the required conditions;
- repair a path with D^* Lite instead of replanning from scratch, and measure when that is worth it;
- represent and detect the conflicts of a multi-agent plan, and solve the problem optimally with conflict-based search or approximately, with a guaranteed bound, with ECBS;
- explain velocity obstacles, reciprocal velocity obstacles and ORCA using relative position and velocity, and use them to keep drones apart;

- track an unknown drone from noisy observations with a Kalman filter, and compare a constant-velocity predictor with a learned one;
- trigger local avoidance from a time-to-collision or safety-horizon rule, return to the global route, or replan;
- enforce formation and communication-range constraints, and
- evaluate a hybrid planner with reproducible metrics and controlled scenarios.

How the book is organised

Part I, Foundations.

?? states the problem and previews the hybrid architecture that the whole book builds towards. ?? is a toolbox: graphs, grids, configuration space, time-indexed paths, simple motion models, the geometry of moving discs, Gaussians and priority queues.

Part II, Single-Agent Graph Search.

Dijkstra's algorithm (??), A^* with admissible and consistent heuristics and its space-time version (??), incremental replanning with LPA^* and D^* Lite (??) and anytime search with ARA^* (??).

Part III, Multi-Agent Path Finding.

The problem and its conflicts (??), prioritized planning (??), conflict-based search (??), bounded-suboptimal search with ECBS (??), and the coupled and constructive alternatives M^* , Push-and-Swap and Push-and-Rotate (??).

Part IV, Local and Reactive Collision Avoidance.

Velocity obstacles (??), reciprocal avoidance with RVO and ORCA (??), the dynamic window approach (??) and artificial potential fields ([Chapter 1](#)).

Part V, Sampling-Based Motion Planning.

RRT (??) and the asymptotically optimal RRT^* with informed sampling (??).

Part VI, Tracking and Prediction.

The Kalman filter (??), its nonlinear relatives and the particle filter (??), and trajectory prediction from constant-velocity baselines to LSTMs and Transformers (??).

Part VII, Optimization and Control.

Model predictive control (??), mixed-integer linear programming (??) and consensus and formation control (??).

Part VIII, Putting It Together.

The hybrid collision-avoidance architecture (??) and how to evaluate it (??).

How to read it

Read the chapters in order if you can: each part builds on the previous ones, and the notation is introduced once. If you are short of time, the training plan's reading order is a good fast path: ?? (A^*), ?? (D^* Lite), ?? (CBS), ?? (the MAPF problem), ???? (velocity obstacles and ORCA), ?? (DWA), ???? (RRT and RRT^*), ?? (MPC) and ?? (trajectory prediction), then ??. Every chapter ends with a box that says where its algorithm sits in the hybrid system, so you can also read by *layer*: the global planner (Parts II and III), the local safety layer (Part IV), the prediction layer (Part VI) and the constraints (Part VII).

Conventions

Every chapter opens with a box of *learning objectives* and closes with a *summary* box and exercises. Inside a chapter you will meet five kinds of coloured boxes: *key idea* (green) states the one thing to remember; *in the drone system* (purple) says how the algorithm is used in the hybrid architecture; *pitfall* (orange) warns about a mistake that is easy to make; *historical note* (grey) gives context; and plain notes carry their own titles. Definitions, examples, theorems and remarks share one counter per chapter, so “Example 4.3” comes after “Definition 4.2”. Exercises are marked with one to three stars (★ routine, ★★ needs thought, ★★★ a small project), and hints or solutions for a selection of them are in ???. Where the training plan assigns a priority to an algorithm it is shown as a tag such as Essential. Pseudocode is written in a uniform style with numbered lines; the text refers to line numbers. Vectors are bold ($\mathbf{p}$, $\mathbf{v}$), matrices are bold capitals ($\mathbf{A}$), and the full notation is listed in the table that follows this preface.

The code

Every algorithm in the book comes with a Python implementation in the `code/` folder of the L^AT_EX project. The implementations are deliberately simple—plain Python and NumPy, a few hundred lines each, no framework—because the goal is that you can read them in one sitting and then write your own. Each file runs a self-test when executed (`python3 code/ch04_astar.py`), and the numbers quoted in the worked examples were produced by that code. The figures that show algorithm output were generated by the scripts in `code/figures/` from fixed random seeds, so you can regenerate and modify them. The listings in the text are excerpts; the files contain the complete programs.

Sources

None of the algorithms in this book is new. Each chapter cites the paper in which its algorithm was introduced and a textbook that treats it at greater length, and the *further reading* paragraph at the end of every chapter points to the developments that the chapter leaves out. The bibliography lists all sources. Where the book simplifies an algorithm for teaching, it says so and names the full version.

Errata

The manuscript was reviewed in several rounds for technical accuracy, completeness against the training plan, and clarity. Mistakes will nevertheless remain; if you find one, the L^AT_EX sources and code are distributed with the book precisely so that it can be corrected.

Notation

This book draws on eight fields that grew up apart, and their alphabets collide. The tables below list every symbol used in more than one place, say what it means, and name the chapter that introduces it. Where a letter really does carry two meanings—**Q** and **R** are the clearest case—the table says so rather than pretend otherwise.

Conventions

Fonts. Scalars are italic (v, t, c, λ), vectors are bold lower case (**p**, **v**, **x**, **u**, **z**) and matrices are bold upper case (**A**, **P**, **Σ**). The zero vector is **0**, the identity matrix **I** and the vector of ones **1**; their sizes follow from the context. Sets and spaces are calligraphic ($\mathcal{C}$, $\mathcal{C}_{\text{free}}$, $\mathcal{C}_i$, $\mathcal{W}$, $\mathcal{N}$), with the one classical exception that the vertex and edge sets of a graph keep the plain letters V and E ; data structures are small capitals (OPEN, CLOSED, FOCAL) and names taken from code are typeset as **this**. $\mathbf{A}^\top$ is the transpose and $\mathbf{A}^{-1}$ the inverse; $\|\cdot\|$ is the Euclidean norm unless a subscript says otherwise, and $|\cdot|$ is an absolute value or, for a finite set, its number of elements.

Indices. A subscript k or t is a time index: $\mathbf{x}_k$ is the state at step k and $\pi[t]$ the vertex occupied at step t . A subscript i or j is an agent index: π_i is the path of agent a_i , $\mathbf{p}_i$ its position, r_i its radius. When both are needed the agent moves up and the time stays down, as in $\mathbf{p}_k^i$; such a superscript is a label, never an exponent, while $\mathbf{A}^k$ is a power. Parenthesised superscripts count samples: $\mathbf{x}^{(i)}$ is the i -th particle or sigma point. Search chapters count time in unit steps, $t = 0, 1, 2, \dots$; control and estimation chapters count steps of the sampling interval, so step k is the instant $k \Delta t$.

Decorations. A star marks an optimal quantity (g^* , h^* , C^* , J^*), a hat an estimate from a filter or a predictor ($\hat{\mathbf{x}}_k$, $\hat{\mathbf{p}}_{T+h}$), a bar an average ($\bar{x}$), and a superscript minus a *prediction* made before the current measurement has been used ($\hat{\mathbf{x}}_k^-$, $\mathbf{P}_k^-$).

General mathematics

Symbol	Meaning	Introduced in
$\mathbb{R}, \mathbb{N}, \mathbb{Z}$	reals, naturals including 0, integers; $\mathbb{R}^n$ holds the n -vectors	??
x , A	vector (bold lower case), matrix (bold upper case)	??
0 , 1 , I	zero vector, vector of ones, identity matrix	??
$\mathbf{A}^\top$, $\mathbf{A}^{-1}$	transpose and inverse of a matrix	??
$\ \mathbf{x}\ $, $ a $	Euclidean norm; absolute value, or size of a finite set	??
$\mathbf{a} \cdot \mathbf{b}$, $\mathbf{a} \times \mathbf{b}$	dot and cross product; $\mathbf{a}^\perp = (-a_y, a_x)$ is a quarter turn	??
$A \oplus B$	Minkowski sum; inflating obstacles by the drone radius	??
arg min, arg max	the argument that minimises or maximises a function	??
diag($\cdot$), tr($\cdot$)	diagonal matrix built from a list; trace of a matrix	??
$\mathcal{O}(f(n))$	asymptotic upper bound in the size of the input	??
$\hat{x}$, $\bar{x}$, x^* , x^-	estimate, average, optimal value, prediction before the update	??

Graphs, grids and search

Symbol	Meaning	Introduced in
$G = (V, E)$	graph with vertex set V and edge set E ; G_4 , G_8 are grids	??
$c(u, v)$	cost of the edge (u, v) , also written w ; costs are non-negative	??
$\text{Succ}(u)$, $\text{Pred}(u)$	successors and predecessors; b is the branching factor	??
s , g	start and goal vertex; the goal is t in ?? and γ from ?? on	??
π , $\text{cost}(\pi)$, $L(\pi)$	a path, its cost, and its Euclidean length	??
$\text{dist}(u, v)$	shortest-path distance; written $\delta(s, v)$ in ??	??
(v, t) , T	space-time state (vertex v at step t); planning horizon	??
$g(n)$, $g^*(n)$	best cost from s to node n found so far; its optimum	??
$h(n)$, $h^*(n)$	heuristic estimate of the cost to go; the true cost to go	??
$f(n)$, C^*	priority $g + h$; optimal solution cost $C^* = h^*(s)$	??
OPEN, CLOSED	the frontier (a priority queue) and the expanded nodes	??
FOCAL, $f_{\min}$	focal band $\{n \in \text{OPEN} : f(n) \leq w f_{\min}\}$; $f_{\min}$ bounds C^*	??
$w \geq 1$	suboptimality factor of weighted A^* , focal search and ECBS	??
ε	inflation factor of ARA^* ; $w = 1 + \varepsilon$ in A^*_ε	??
$\text{rhs}(s)$	one-step look-ahead value; $g(s) = \text{rhs}(s)$ is local consistency	??
$\text{key}(s)$, k_m	two-component priority and key modifier of D^* Lite	??

Multi-agent path finding

Symbol	Meaning	Introduced in
k , a_i	number of agents; agent i , with start s_i and goal g_i or γ_i	??
π_i , $\pi_i[t]$	path of a_i and the vertex it occupies at step t	??
$\text{cost}(\pi_i)$, T_i	time of final arrival at the goal; last stored index of π_i	??
$\Pi = (\pi_1, \dots, \pi_k)$	a plan: one time-indexed path per agent	??
$\text{SoC}(\Pi)$	sum of costs: the total of $\text{cost}(\pi_i)$ over all agents	??
$\text{MS}(\Pi)$	makespan: the largest $\text{cost}(\pi_i)$, $\text{MS} \leq \text{SoC} \leq k \text{MS}$	??
$\langle a_i, a_j, v, t \rangle$	vertex conflict: both agents occupy v at time t	??
$\langle a_i, a_j, u, v, t \rangle$	edge or swapping conflict on $\{u, v\}$ between t and $t + 1$	??
$\langle a_i, v, t \rangle$	vertex constraint: a_i may not occupy v at time t	??
$\mathcal{C}_i$, N	constraints on a_i ; constraint-tree node $N.C$, $N.\pi$, $N.\text{cost}$	??
σ	priority order: a permutation of the agents, $\sigma(1)$ first	??
h_c	number of remaining conflicts, the FOCAL heuristic of ECBS	??
$C(v)$, $\Psi(v, v')$	collision set of a joint vertex and of a joint move; policy ϕ_i	??

Geometry and motion

Symbol	Meaning	Introduced in
$\mathcal{W}, \mathcal{O}$	workspace and the obstacle region inside it	??
$\mathcal{C}, \mathcal{C}_{\text{free}}, \mathcal{C}_{\text{obs}}$	configuration space, its free part, its obstacle region	??
$D(\mathbf{c}, r)$	closed disc (in 3D, ball) of radius r around $\mathbf{c}$	??
$r_i, R = r_A + r_B$	radius of an agent; collision distance of a pair	??
$\mathbf{p}, \mathbf{v}, \mathbf{a}$	position, velocity, acceleration; limits v_{max} and a_{max}	??
Δt	time step (sampling interval), in seconds	??
$\mathbf{x}, \mathbf{u}$	state and control input of a motion model	??
$\mathbf{A}, \mathbf{B}$	state-transition and input matrix of the double integrator	??
$d_{ij}(t), d_{\min}$	separation of two agents; its minimum over the execution	??
$d_{\text{stop}}(v)$	stopping distance $v^2/(2a_{\text{max}})$ of a braking drone	??
t^*, t_c	time of closest approach and time to collision of $\mathbf{p}_{\text{rel}}, \mathbf{v}_{\text{rel}}$	??
τ	horizon of the reactive layer; a time to collision in ??	??
$VO_{A B}, VO_{A B}^\tau$	velocity obstacle and its truncation at τ ; $CC_{A B}$ is its cone	??
$RVO_{A B}$	reciprocal velocity obstacle, apex at $\frac{1}{2}(\mathbf{v}_A + \mathbf{v}_B)$	??
$ORCA_{A B}^\tau$	the ORCA half-plane of velocities A may choose	??
$\mathbf{u}, \mathbf{n}$	smallest change of the relative velocity and the outward normal	??
$\mathbf{v}^{\text{pref}}, \mathbf{v}^{\text{new}}$	preferred velocity and the velocity finally commanded	??
V_s, V_a, V_d, V_r	possible, admissible, dynamic-window and resulting velocities	??
$U(\mathbf{p}), \rho, \rho_0$	potential field; clearance and influence distance	Chapter 1
$\mathcal{X}_{\text{free}}, \mathcal{X}_{\text{obs}}$	free and blocked parts of the state space, the $\mathcal{C}_{\text{free}}, \mathcal{C}_{\text{obs}}$ above	??
$T = (V, E), \eta$	tree grown by RRT and its step size; Nearest, Steer extend it	??

Probability and filtering

Symbol	Meaning	Introduced in
$\mathbb{P}(\cdot), p(\mathbf{x})$	probability of an event; density of a random vector	??
$\mathbb{E}[\cdot], \text{Var}, \text{Cov}$	expectation, variance, covariance	??
$\mathcal{N}(\mu, \Sigma)$	Gaussian of mean μ , covariance Σ ; σ_i, ρ_{ij} are its spreads	??
$\mathcal{E}_k$	$k\sigma$ covariance ellipse: Mahalanobis distance k from the mean	??
$\mathbf{x}_k, \mathbf{z}_k$	state and measurement at step k ; $\mathbf{z}_{1:k}$ all up to k	??
$\hat{\mathbf{x}}_k^-, \mathbf{P}_k^-$	predicted mean and covariance, before $\mathbf{z}_k$ is used	??
$\hat{\mathbf{x}}_k, \mathbf{P}_k$	updated (posterior) mean and covariance	??
$\mathbf{F}_k$	state-transition matrix, or the Jacobian $\partial f / \partial \mathbf{x}$	??
$\mathbf{H}_k$	measurement matrix, or the Jacobian $\partial h / \partial \mathbf{x}$	??
$\mathbf{Q}_k, \mathbf{R}_k$	process-noise and measurement-noise covariance	??
$\mathbf{w}_k, \mathbf{v}_k$	process noise and measurement noise, of covariance $\mathbf{Q}_k, \mathbf{R}_k$	??
$\mathbf{y}_k, \mathbf{S}_k$	innovation $\mathbf{z}_k - h(\hat{\mathbf{x}}_k^-)$ and its covariance	??
$\mathbf{K}_k$	Kalman gain $\mathbf{P}_k^- \mathbf{H}_k^T \mathbf{S}_k^{-1}$	??
$\mathcal{X}_i, \mathbf{x}^{(i)}, w^{(i)}$	sigma point; particle i and its weight	??
$T_{\text{obs}}, T_{\text{pred}}$	observation window and prediction horizon; h is a horizon step	??
ADE, FDE	average and final displacement error; minADE _K over K samples	??
$\mathbf{Q}, \mathbf{K}, \mathbf{V}$	query, key and value matrices of attention; this meaning only here	??

Two warnings about letters. $\mathbf{Q}$ and $\mathbf{R}$ are *noise covariances* in Part VI, where they say how far the motion model and the sensor may be trusted, and *cost weights* in Part VII, where they say how much a tracking error and a control effort cost. The two roles are unrelated, they carry different units, and no equation in this book uses both at once; ?? repeats the warning where the weights appear. Second, this book follows the estimation literature and writes $\mathbf{F}$ for the state-transition matrix and $\mathbf{H}$ for the measurement matrix of a filter, where control

texts usually write $\mathbf{A}$ and $\mathbf{C}$. The plant models of ?? and the controller of ?? do use $\mathbf{A}$ and $\mathbf{B}$, because there they describe a vehicle and not a filter.

Optimization and control

Symbol	Meaning	Introduced in
J, J^*	objective of an optimization problem and its optimal value	??
subject to	introduces the constraints of an optimization problem	??
N	prediction horizon in steps of Δt ; $k = 0$ is the present	??
$\mathbf{x}_k^{\text{ref}}$	reference state at step k of the horizon	??
$\mathbf{Q} \succeq \mathbf{0}, \mathbf{R} \succ \mathbf{0}$	weights on the state error and on the control effort	??
$\mathbf{P} \succeq \mathbf{0}$	terminal cost weight on the last predicted state	??
$\mathbf{S}_x, \mathbf{S}_u$	prediction matrices that stack the horizon into one map	??
$\mathbf{z}$	stacked decision variable of the resulting QP or LP	??
$\mathbf{H}, \mathbf{f}, \mathbf{G}$	Hessian, linear term and constraint matrix, $\mathbf{l} \leq \mathbf{G}\mathbf{z} \leq \mathbf{u}$	??
$\mathbf{y}, \rho$	Lagrange multipliers and penalty parameter of the ADMM solver	??
s_k	slack variable that softens a constraint at step k	??
ε	tolerance of a numerical solver (residuals, integrality)	??
$\mathbf{c}, \mathbf{A}, \mathbf{b}$	LP data: $\min \mathbf{c}^T \mathbf{z}$ over $\mathbf{A}\mathbf{z} \leq \mathbf{b}$; P is the feasible polyhedron	??
$z_j \in \mathbb{Z}, b$	integer variable and binary variable of a mixed-integer program	??
M	big- M constant: $b = 1$ switches an inequality off	??

Networks and consensus

Symbol	Meaning	Introduced in
$G = (V, E)$	communication graph: the drones and the pairs that can talk	??
N_i	the neighbours of drone i in that graph	??
$\mathbf{A} = (a_{ij}), \mathbf{D}$	adjacency matrix; degree matrix $\text{diag}(d_1, \dots, d_n)$	??
$\mathbf{L} = \mathbf{D} - \mathbf{A}$	graph Laplacian; symmetric, $\mathbf{L}\mathbf{1} = \mathbf{0}$	??
λ_2, λ_n	algebraic connectivity (Fiedler value); largest eigenvalue of $\mathbf{L}$	??
$\bar{\mathbf{x}}, \delta$	average of the states; disagreement vector $\mathbf{x} - \bar{\mathbf{x}}\mathbf{1}$	??
ε	step size of discrete consensus, $\mathbf{x}_{k+1} = (\mathbf{I} - \varepsilon\mathbf{L})\mathbf{x}_k$	??
R_{comm}	communication range: $\{i, j\} \in E$ iff $\ \mathbf{p}_i - \mathbf{p}_j\ \leq R_{\text{comm}}$	??
$\mathbf{o}_i, \mathbf{d}_{ij}$	formation offset of drone i ; desired displacement $\mathbf{o}_j - \mathbf{o}_i$	??
e_F, e_{max}	formation error (RMS displacement violation) and its tolerance	??
k_p, k_v, k_d	position, velocity and damping gains of second-order consensus	??

Part I

Foundations

Part II

Single-Agent Graph Search

Part III

Multi-Agent Path Finding

Part IV

Local and Reactive Collision Avoidance

Artificial Potential Fields

Learning objectives

- Explain how a potential that is low at the goal and high near obstacles turns navigation into gradient descent, and write down Khatib's attractive and repulsive potentials with their forces.
- Derive the forces from the potentials, evaluate them by hand at a given point, and trace the discrete-time controller with a velocity limit.
- Reproduce the four classic failure modes – local minima, goals that cannot be reached next to obstacles, oscillation, and blocked passages – and explain each with a formula, not only with a picture.
- Apply the standard remedies, state what navigation functions and harmonic potentials guarantee, and use a potential field as a local layer under a global plan.
- Extend the field to several drones with inter-agent repulsion and relate it to Reynolds's flocking rules and to consensus.

1.1 Why this matters for a drone swarm

A drone of the swarm is following the path that the global planner of ?? assigned to it when a bird, or a drone that is not part of the swarm, appears three metres ahead. The on-board computer has perhaps ten milliseconds per control cycle, and in that time it must produce a velocity command that moves the drone away from the intruder and, as soon as it can, back towards its goal. The oldest and simplest answer is the **artificial potential field** (APF) of Khatib [6]: imagine a landscape over the map that slopes down towards the goal and rises steeply around every obstacle, and let the drone roll downhill. The command is the negative gradient of that landscape; per obstacle it costs one distance query and one square root, with no search, no queue and no optimisation.

The method has an equally old list of defects. Five years after Khatib, Koren and Borenstein [8] catalogued the situations in which a potential field traps the robot, refuses to let it through a doorway, or makes it oscillate against a wall. Every one of these failures has a precise mathematical cause, and in this chapter you will produce each of them on purpose, in a few lines of Python, and see what the cause is. That is what the Week 7 milestone of the study plan (??) asks for: *know when local methods work and when they fail*. The week pairs this chapter with the dynamic window approach (DWA) of ??, and its coding exercise (Exercise 1.9) asks you to implement one of the two and to construct its failure cases. In the hybrid architecture of ?? the safety layer is ORCA or the dynamic window approach, not a potential field; Section 1.11 explains why, and where potential fields survive.

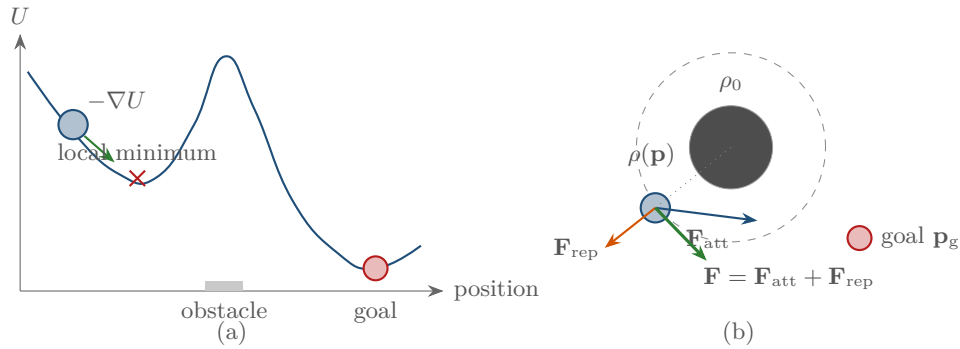


Figure 1.1. (a) A one-dimensional potential: the goal is the global minimum, the obstacle a peak, and the drone follows the negative gradient. The dip to the left of the obstacle is a local minimum; a drone released there never leaves it. (b) In the plane the attractive force $\mathbf{F}_{\text{att}}$ points at the goal, the repulsive force $\mathbf{F}_{\text{rep}}$ points away from the nearest point of the obstacle, and the command is their sum; the repulsion is zero outside the dashed circle of radius ρ_0 .

The chapter proceeds as follows. Sections 1.2 and 1.3 give the picture, the potentials and the derivations; Sections 1.4 and 1.5 state the controller and evaluate the forces by hand; Section 1.6 collects what can be proved. Section 1.7 is the heart of the chapter: the four failure modes, each with a runnable example, and the Koren–Borenstein critique. Sections 1.8 and 1.9 present the remedies, the basin experiment and the extension to swarms, and Sections 1.10 and 1.11 cover the implementation and the place of the method in the drone system.

1.2 The idea in plain words

Think of a marble on a hilly table (Figure 1.1a). The table is shaped so that the goal is the lowest point and every obstacle sits on top of a steep hill. Released anywhere, the marble rolls downhill: away from the hills, towards the goal. The height of the table is the **potential** $U(\mathbf{p})$, a scalar function of the position; the direction of steepest descent at $\mathbf{p}$ is the negative gradient $-\nabla U(\mathbf{p})$, and this vector is the **force** that the field exerts on the drone. The drone does not plan a path; at every control cycle it evaluates one vector and follows it.

The potential is built from two kinds of terms (Figure 1.1b). The *attractive* term is a bowl centred at the goal; its gradient points straight at the goal, and the further away the drone is, the harder it pulls. Each *repulsive* term is a wall around one obstacle; it is zero beyond an *influence distance* ρ_0 , grows as the drone approaches the obstacle, and becomes infinite on its boundary, so that the drone can never touch it. Forces are vectors, so they add: the command is the sum of the attraction and of the repulsions of every obstacle within range. That is the whole method. Everything else in this chapter is about the two words that the picture hides. Downhill is a *local* notion: the marble finds the lowest point of the valley it is in, which need not be the goal. And the drone is not a marble in continuous time but a computer that takes finite steps.

Key idea

Add a bowl at the goal to a wall around every obstacle and move against the gradient of the sum. The command is cheap and smooth, and in continuous time it never touches an obstacle. But the gradient only sees the slope under the drone's feet: the drone stops wherever attraction and repulsion cancel, and that point is the goal only if you were lucky with the scene.

1.3 Problem statement and notation

Definition 1.1 (Potential-field navigation problem). The drone is a point moving in a workspace $\mathcal{W} \subseteq \mathbb{R}^2$ (or $\mathbb{R}^3$); its physical radius is absorbed by inflating the obstacles, as in ???. The obstacles $O_1, \dots, O_m$ are closed sets, the free space is $\mathcal{C}_{\text{free}} = \mathcal{W} \setminus \bigcup_i O_i$, and the goal is a point $\mathbf{p}_g \in \mathcal{C}_{\text{free}}$. The **clearance** to obstacle i is the distance

$$\rho_i(\mathbf{p}) = \min_{\mathbf{o} \in O_i} \|\mathbf{p} - \mathbf{o}\|, \quad (1.1)$$

and $\nabla \rho_i(\mathbf{p})$ is its gradient, the unit vector that points from the closest point of O_i towards $\mathbf{p}$ (defined wherever that closest point is unique). A **potential** is a continuously differentiable function $U: \mathcal{C}_{\text{free}} \rightarrow \mathbb{R}_{\geq 0}$; its **force** is $\mathbf{F}(\mathbf{p}) = -\nabla U(\mathbf{p})$. The task is to choose U so that a drone that follows $\mathbf{F}$ from a given start reaches $\mathbf{p}_g$ without leaving $\mathcal{C}_{\text{free}}$.

For a disc with centre $\mathbf{c}$ and radius r the clearance is $\rho(\mathbf{p}) = \|\mathbf{p} - \mathbf{c}\| - r$ and $\nabla \rho = (\mathbf{p} - \mathbf{c}) / \|\mathbf{p} - \mathbf{c}\|$; for a segment it is the point-to-segment distance of ??? and $\nabla \rho$ points away from the closest point. The code of this chapter uses discs; any convex obstacle is handled by the same two quantities, a distance and a unit vector.

Definition 1.2 (Attractive potential). Let $d(\mathbf{p}) = \|\mathbf{p} - \mathbf{p}_g\|$ and $k_{\text{att}} > 0$. The **quadratic attractive potential** and its force are

$$U_{\text{att}}(\mathbf{p}) = \frac{1}{2} k_{\text{att}} d(\mathbf{p})^2, \quad \mathbf{F}_{\text{att}}(\mathbf{p}) = -\nabla U_{\text{att}}(\mathbf{p}) = -k_{\text{att}} (\mathbf{p} - \mathbf{p}_g). \quad (1.2)$$

The **conic** potential is $U_{\text{att}} = k_{\text{att}} d^* d$ for a constant $d^* > 0$, with a force of constant magnitude $k_{\text{att}} d^*$. The **hybrid** potential is quadratic for $d \leq d^*$ and conic beyond:

$$U_{\text{att}}(\mathbf{p}) = \begin{cases} \frac{1}{2} k_{\text{att}} d^2 & d \leq d^*, \\ k_{\text{att}} d^* d - \frac{1}{2} k_{\text{att}} (d^*)^2 & d > d^*, \end{cases} \quad \mathbf{F}_{\text{att}}(\mathbf{p}) = \begin{cases} -k_{\text{att}} (\mathbf{p} - \mathbf{p}_g) & d \leq d^*, \\ -k_{\text{att}} d^* (\mathbf{p} - \mathbf{p}_g) / d & d > d^*. \end{cases} \quad (1.3)$$

The gradient of $\frac{1}{2}d^2$ is $\mathbf{p} - \mathbf{p}_g$ and the gradient of d is the unit vector $(\mathbf{p} - \mathbf{p}_g)/d$, which gives the two forces. Each form has one defect: the quadratic force grows without bound with the distance, so that a drone far from its goal but next to an obstacle is pulled through the repulsion, while the conic force is bounded but discontinuous at the goal, where the unit vector flips, so a drone under it chatters across the goal. The hybrid form takes the bounded force far away and the smooth bowl near the goal; the constant $-\frac{1}{2}k_{\text{att}}(d^*)^2$ makes U_{att} continuous at $d = d^*$ and both branches give a force of magnitude $k_{\text{att}}d^*$ there (Exercise 1.2); it is the form recommended by Choset et al. [3]. Clipping the quadratic force at a speed v_{max} , as the controller of Section 1.4 does, is the same as the hybrid potential with $d^* = v_{\text{max}}/k_{\text{att}}$, so the examples of this chapter use the quadratic bowl with $k_{\text{att}} = 1$ and $v_{\text{max}} = 1$.

Definition 1.3 (Repulsive potential, Khatib 1986). Let $k_{\text{rep}} > 0$ and let $\rho_0 > 0$ be the **influence distance**. The **repulsive potential** of obstacle i and its force are

$$U_{\text{rep},i}(\mathbf{p}) = \begin{cases} \frac{1}{2} k_{\text{rep}} \left(\frac{1}{\rho_i(\mathbf{p})} - \frac{1}{\rho_0} \right)^2 & \rho_i(\mathbf{p}) \leq \rho_0, \\ 0 & \rho_i(\mathbf{p}) > \rho_0, \end{cases} \quad (1.4)$$

$$\mathbf{F}_{\text{rep},i}(\mathbf{p}) = -\nabla U_{\text{rep},i}(\mathbf{p}) = \begin{cases} k_{\text{rep}} \left(\frac{1}{\rho_i(\mathbf{p})} - \frac{1}{\rho_0} \right) \frac{1}{\rho_i(\mathbf{p})^2} \nabla \rho_i(\mathbf{p}) & \rho_i(\mathbf{p}) \leq \rho_0, \\ \mathbf{0} & \rho_i(\mathbf{p}) > \rho_0. \end{cases} \quad (1.5)$$

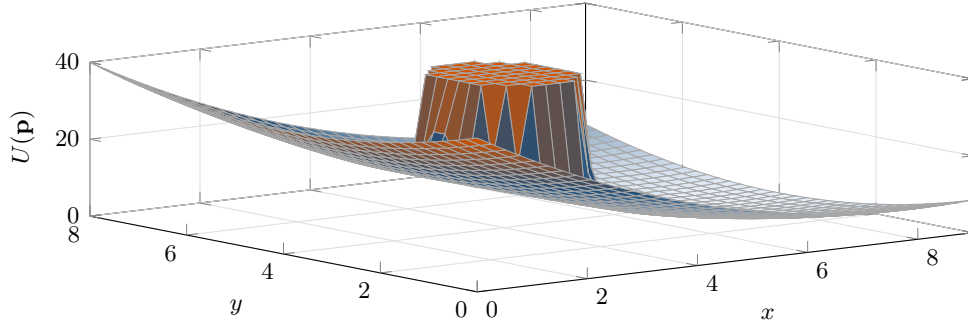


Figure 1.2. The total potential U of the one-disc scene (goal at $(8, 4)$, disc of radius 1 at $(4, 4)$, $k_{\text{att}} = k_{\text{rep}} = 1$, $\rho_0 = 2$), clipped at $U = 40$. The quadratic bowl is centred at the goal; the repulsive term is zero beyond $\rho_0 = 2$ from the disc and rises to infinity at its boundary. Notice how gentle the bowl is compared with the chimney: close to the obstacle the attraction is negligible, far from it the obstacle is invisible.

The derivation is one application of the chain rule. Inside the influence region write $g(\mathbf{p}) = 1/\rho_i(\mathbf{p}) - 1/\rho_0$, so that $U_{\text{rep},i} = \frac{1}{2}k_{\text{rep}}g^2$ and $\nabla U_{\text{rep},i} = k_{\text{rep}}g\nabla g$. Since $\nabla(1/\rho_i) = -\rho_i^{-2}\nabla\rho_i$,

$$\nabla U_{\text{rep},i} = k_{\text{rep}} \left(\frac{1}{\rho_i} - \frac{1}{\rho_0} \right) \left(-\frac{1}{\rho_i^2} \right) \nabla \rho_i,$$

and the force is the negative of this. It points along $\nabla\rho_i$, straight away from the closest point of the obstacle, and its magnitude $k_{\text{rep}}(1/\rho_i - 1/\rho_0)/\rho_i^2$ behaves like k_{rep}/ρ_i^3 as $\rho_i \rightarrow 0$: the wall is infinitely high and infinitely steep at the obstacle boundary. At $\rho_i = \rho_0$ both the potential and the force are zero and join continuously with the zero outside. Khatib called this the FIRAS potential (*force involving artificial repulsion from the surface*); the subtraction of $1/\rho_0$ limits the influence of an obstacle to its neighbourhood.

Definition 1.4 (Total potential, critical points). The total potential and the total force are

$$U(\mathbf{p}) = U_{\text{att}}(\mathbf{p}) + \sum_{i=1}^m U_{\text{rep},i}(\mathbf{p}), \quad \mathbf{F}(\mathbf{p}) = \mathbf{F}_{\text{att}}(\mathbf{p}) + \sum_{i=1}^m \mathbf{F}_{\text{rep},i}(\mathbf{p}). \quad (1.6)$$

A point with $\mathbf{F}(\mathbf{p}) = \mathbf{0}$ is a **critical point** of U . It is a **local minimum** if U does not decrease in any direction from it, and a **saddle** if U decreases in some directions and increases in others. The **basin** of a local minimum is the set of starts from which the descent ends there.

Figure 1.2 shows the total potential of the scene used throughout the chapter: one disc of radius 1 centred at $(4, 4)$, the goal at $(8, 4)$, $k_{\text{att}} = k_{\text{rep}} = 1$ and $\rho_0 = 2$. What the surface does not show at this scale is the feature that matters most: on the line from the far side of the disc to the goal, the slopes of the bowl and of the chimney cancel at one point, a critical point that is not the goal (Section 1.7.1).

1.4 The algorithm

The method has two parts: the evaluation of the force at a point (Algorithm 1.1) and the loop that turns the force into motion (Algorithm 1.2). Algorithm 1.1 is Equations (1.3), (1.5) and (1.6) written out, with one addition at line 12: the optional factor d^n of Ge and Cui [5] that cures the second failure mode of Section 1.7. With $n = 0$ the line is skipped and the function computes Khatib's original force.

Algorithm 1.1. The potential-field force at a point: hybrid attraction, Khatib repulsion, optional GNRON factor

Input: position $\mathbf{p}$, goal $\mathbf{p}_g$, obstacles $O_1, \dots, O_m$ with distance queries; gains $k_{\text{att}}, k_{\text{rep}}$, influence distance ρ_0 , switch distance d^* (∞ for a purely quadratic bowl), exponent $n \geq 0$ ($n = 0$ for Khatib's original)

Output: the force $\mathbf{F}(\mathbf{p}) = -\nabla U(\mathbf{p})$

```

1 Function ApfForce( $\mathbf{p}$ ,  $\mathbf{p}_g$ ,  $O_{1..m}$ ):
2    $\mathbf{d} \leftarrow \mathbf{p} - \mathbf{p}_g$ ;  $d \leftarrow \|\mathbf{d}\|$ 
3   if  $d \leq d^*$  then
4      $\mathbf{F} \leftarrow -k_{\text{att}} \mathbf{d}$                                 // quadratic bowl
5   else
6      $\mathbf{F} \leftarrow -k_{\text{att}} d^* \mathbf{d}/d$                         // conic far away: constant magnitude
7   foreach obstacle  $O_i$  do
8      $(\rho, \nabla \rho) \leftarrow \text{Distance}(O_i, \mathbf{p})$            // clearance and unit vector away from  $O_i$ 
9     if  $\rho < \rho_0$  then
10       $\mathbf{F}_i \leftarrow k_{\text{rep}} (1/\rho - 1/\rho_0) \rho^{-2} \nabla \rho$ 
11      if  $n \geq 1$  then
12         $\mathbf{F}_i \leftarrow d^n \mathbf{F}_i - \frac{n}{2} k_{\text{rep}} (1/\rho - 1/\rho_0)^2 d^{n-1} \mathbf{d}/d$  // GNRON correction,
13         $\mathbf{F} \leftarrow \mathbf{F} + \mathbf{F}_i$                                Section 1.7.2
14  return  $\mathbf{F}$ 

```

Walkthrough. Line 4 of Algorithm 1.1 is the attraction; with $d^* = \infty$ it is always the quadratic branch. Line 8 is the only geometric operation of the whole method, a query for the clearance to one obstacle and the unit vector away from its closest point; an obstacle further away than ρ_0 contributes nothing, so a spatial index that returns only the obstacles within ρ_0 makes the loop independent of the size of the map. Line 13 adds the vectors, which is where all the trouble of Section 1.7 originates, because a sum of vectors can be zero. Algorithm 1.2 is explicit Euler integration of $\dot{\mathbf{p}} = \mathbf{F}(\mathbf{p})$ with step Δt (line 12), guarded by three tests: the goal tolerance ε_g (line 4), needed because the quadratic bowl makes the drone approach the goal exponentially and never arrive exactly (Exercise 1.2); the collision test (line 6), which cannot fire in continuous time (Theorem 1.7) but can in discrete time (Section 1.7.3); and the **local-minimum detector** of line 8, which declares the drone stuck if it has moved less than δ during the last W steps – the only way a gradient method can notice a local minimum, since the force there is zero and the drone simply stops.

Line 11 is the **velocity limit**. The force has the units of a velocity only by convention: Khatib's original applied $\mathbf{F}$ to a mass with viscous damping, $m\dot{\mathbf{p}} = \mathbf{F} - b\dot{\mathbf{p}}$, while almost every mobile-robot implementation uses the first-order form $\mathbf{v} = \mathbf{F}$ and sends $\mathbf{v}$ to the velocity controller of the vehicle. The magnitude of $\mathbf{F}$ is then arbitrary and can be enormous, k_{rep}/ρ^3 at a clearance of a few centimetres. The clip keeps the direction, which is the information the field carries, caps the speed at what the drone can fly, and bounds every step by $\Delta t v_{\text{max}}$, which Section 1.7.3 shows to be the difference between a controller that chatters and one that crashes. The chapter uses $k_{\text{att}} = k_{\text{rep}} = 1$, $\rho_0 = 2$, $v_{\text{max}} = 1$, $\Delta t = 0.01$, $\varepsilon_g = 0.05$, $W = 100$ and $\delta = 10^{-3}$ unless stated otherwise; once the speed is clipped only the ratio of the two gains shapes the trajectory, ρ_0 decides how early an obstacle is felt, and Δt and v_{max} set the step length.

Algorithm 1.2. Gradient-descent controller with a velocity limit and local-minimum detection

Input: start $\mathbf{p}_0$, goal $\mathbf{p}_g$, obstacles $O_{1..m}$; speed limit $v_{\max}$, control period Δt , goal tolerance ε_g , horizon $K_{\max}$, stuck window W (steps) and stuck tolerance δ

Output: the path $\mathbf{p}_0, \mathbf{p}_1, \dots$ and a status in {reached, collision, stuck, timeout}

```

1 Function ApfDescend( $\mathbf{p}_0, \mathbf{p}_g, O_{1..m}$ ):
2   for  $k \leftarrow 0$  to  $K_{\max} - 1$  do
3     if  $\|\mathbf{p}_k - \mathbf{p}_g\| \leq \varepsilon_g$  then
4       return (reached,  $\mathbf{p}_{0..k}$ )
5     if  $\min_i \rho_i(\mathbf{p}_k) \leq 0$  then
6       return (collision,  $\mathbf{p}_{0..k}$ )
7     if  $k \geq W$  and  $\|\mathbf{p}_k - \mathbf{p}_{k-W}\| < \delta$  then
8       return (stuck,  $\mathbf{p}_{0..k}$ )           // no progress: a local minimum
9      $\mathbf{v} \leftarrow \text{ApfForce}(\mathbf{p}_k, \mathbf{p}_g, O_{1..m})$ 
10    if  $\|\mathbf{v}\| > v_{\max}$  then
11       $\mathbf{v} \leftarrow v_{\max} \mathbf{v} / \|\mathbf{v}\|$        // velocity limit: keep the direction, cap the speed
12     $\mathbf{p}_{k+1} \leftarrow \mathbf{p}_k + \Delta t \mathbf{v}$ 
13  return (timeout,  $\mathbf{p}_{0..K_{\max}}$ )

```

1.5 A worked example

Example 1.5 (One drone, one obstacle). The scene is that of [Figure 1.2](#): a disc obstacle of radius 1 centred at $(4, 4)$, the goal $\mathbf{p}_g = (8, 4)$, the gains $k_{\text{att}} = k_{\text{rep}} = 1$ and the influence distance $\rho_0 = 2$, so the repulsion acts within 3 units of the disc centre (the dashed circle in [Figure 1.3](#)). We evaluate the forces at three probe points, $A = (0.5, 4)$ far to the left, $B = (2.5, 4)$ on the axis half a unit from the disc, and $C = (3, 5.3)$ above and to the left of it, and then run [Algorithm 1.2](#) from the start $(0, 4.5)$. The instance is `worked_example()` in `ch15_potential_fields.py`, and every number below is printed by its self-test.

Point A $= (0.5, 4)$. The distance to the disc centre is 3.5, so the clearance is $\rho = 2.5 > \rho_0$ and the repulsion is zero. The attraction is $\mathbf{F}_{\text{att}} = -(\mathbf{p} - \mathbf{p}_g) = -(0.5 - 8, 4 - 4) = (7.5, 0)$, and $U = \frac{1}{2} \cdot 7.5^2 = 28.125$.

Point B $= (2.5, 4)$. Now $\rho = 1.5 - 1 = 0.5$ and $\nabla \rho = (-1, 0)$. The attraction is $(5.5, 0)$. The repulsion has the magnitude $(1/0.5 - 1/2)/0.5^2 = 1.5/0.25 = 6$, so $\mathbf{F}_{\text{rep}} = (-6, 0)$ and the total force is $(-0.5, 0)$: the drone is pushed *back*, away from the goal, although the goal is straight ahead. The potential is $\frac{1}{2} \cdot 5.5^2 + \frac{1}{2} \cdot 1.5^2 = 16.25$. Somewhere between *A* and *B* the horizontal force changes sign, and at that point a drone on the axis stops ([Section 1.7.1](#)).

Point C $= (3, 5.3)$. The offset from the centre is $(-1, 1.3)$, of length $\sqrt{2.69} = 1.6401$, so $\rho = 0.6401$ and $\nabla \rho = (-0.6097, 0.7926)$. The attraction is $(5, -1.3)$. The repulsion has magnitude $(1/0.6401 - 0.5)/0.6401^2 = 2.5928$ and equals $(-1.5805, 2.0547)$; the total force is $(3.4195, 0.7547)$, of magnitude 3.5018. The drone is deflected upwards, over the obstacle, while still making progress towards the goal. [Table 1.1](#) collects the three evaluations.

[Figure 1.3](#) shows the trajectory from $(0, 4.5)$. The drone flies straight until it enters the influence circle, is deflected over the top of the disc, and curves back down to the goal, which it reaches after 1099 steps, that is, 10.99 time units, with a minimum clearance of 0.519. The path is 8.957 units long against a straight-line distance of 8.016; the detour costs about 12 %. The drone flies at the speed limit until it is one unit from the goal; the last stretch is the exponential approach of the quadratic bowl, which is why the goal tolerance is needed. The

Table 1.1. The forces of Example 1.5 at the three probe points of Figure 1.3 ($k_{\text{att}} = k_{\text{rep}} = 1$, $\rho_0 = 2$). At B the repulsion beats the attraction although the goal is straight ahead; at C the two forces are at an angle and their sum turns the drone around the obstacle.

Point	$\mathbf{p}$	ρ	$\mathbf{F}_{\text{att}}$	$\mathbf{F}_{\text{rep}}$	$\mathbf{F}$	U
A	(0.5, 4)	2.5	(7.5, 0)	(0, 0)	(7.5, 0)	28.125
B	(2.5, 4)	0.5	(5.5, 0)	(-6, 0)	(-0.5, 0)	16.250
C	(3, 5.3)	0.6401	(5, -1.3)	(-1.5805, 2.0547)	(3.4195, 0.7547)	13.909

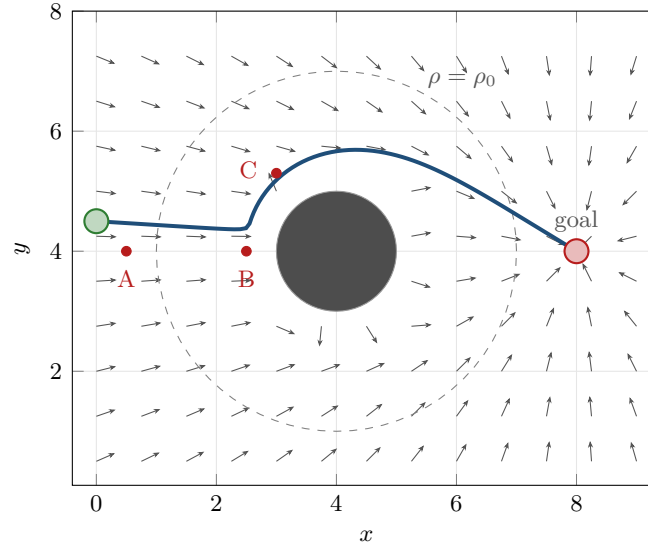


Figure 1.3. The force field of Example 1.5: arrows show the direction of $\mathbf{F}$, the dashed circle is the influence boundary $\rho = \rho_0$, the red dots are the probe points of Table 1.1, and the blue curve is the trajectory from (0, 4.5), which passes the disc with a minimum clearance of 0.519 and reaches the goal after 1 099 steps. Notice the arrows on the axis between $x = 1$ and $x = 3$, which first point right and then left: there the two forces cancel at one point.

same run with the hybrid potential ($d^* = 2$) reaches the goal in 1 171 steps with a larger clearance, 0.695: next to the disc the conic pull has the magnitude $k_{\text{att}}d^* = 2$ instead of about 5, so the repulsion wins more ground. The ratio of the two forces near an obstacle, not their absolute size, shapes the path.

Table 1.2 traces the same run at seven steps. The first rows are the straight approach: the force is long, the speed limit binds, and the command is the unit vector along $\mathbf{F}$. At $k = 300$ the drone is 0.529 from the disc, the repulsion has turned the force upwards, its magnitude has fallen to 2.162, and the command still has length 1. By $k = 800$ the drone is 1.007 from the goal, $\|\mathbf{F}\|$ has dropped to the speed limit, and from there the clip releases: the last stretch is the exponential tail of the bowl, $d(t) = d(8)e^{-k_{\text{att}}(t-8)}$, which needs $\ln(1.007/0.05) = 3.00$ time units to reach the tolerance – the 299 steps that the table shows between the last two rows.

1.6 Properties: continuity, descent, safety, and where the minimum is

A potential field is not a search algorithm, and the usual questions get unusual answers. It is not complete (Section 1.7 exhibits instances with a free path on which it fails) and not optimal in any sense (nothing in it measures path length). Its complexity is its strength: one evaluation of Algorithm 1.1 costs $\mathcal{O}(m)$ distance queries for m obstacles, or $\mathcal{O}(m_{\text{loc}})$ with a

Table 1.2. Trace of [Algorithm 1.2](#) from (0,4.5) in the scene of [Example 1.5](#), printed by `worked_example()`: step k , position, clearance, force, its magnitude, the clipped velocity command $\mathbf{v}_k$ and the distance to the goal. The speed limit binds in every row but the last, so the command is a unit vector while the force is not; it releases inside the last unit, where the drone crosses the goal tolerance $\varepsilon_g = 0.05$ after 1 099 steps.

k	$\mathbf{p}_k$	ρ	$\mathbf{F}(\mathbf{p}_k)$	$\ \mathbf{F}\ $	$\mathbf{v}_k$	$\ \mathbf{p}_k - \mathbf{p}_g\ $
0	(0.000, 4.500)	3.031	(8.000, -0.500)	8.016	(0.998, -0.062)	8.016
100	(0.998, 4.438)	2.034	(7.002, -0.438)	7.016	(0.998, -0.062)	7.016
200	(1.996, 4.377)	1.039	(5.583, -0.298)	5.591	(0.999, -0.053)	6.016
300	(2.713, 4.826)	0.529	(1.104, 1.859)	2.162	(0.510, 0.860)	5.351
500	(4.410, 5.688)	0.737	(3.962, -0.156)	3.965	(0.999, -0.039)	3.967
800	(7.140, 4.524)	2.183	(0.860, -0.524)	1.007	(0.854, -0.520)	1.007
1 099	(7.957, 4.026)	2.957	(0.043, -0.026)	0.050	(0.043, -0.026)	0.050

spatial index that returns the m_{loc} obstacles within ρ_0 , with no map representation, no queue and no memory beyond the obstacle list. What can be proved is what the marble picture promises: the drone always goes downhill, never touches an obstacle in continuous time, and stops at a critical point.

Proposition 1.6 (Continuity of the field). *On the subset of $\mathcal{C}_{\text{free}}$ where every $\nabla\rho_i$ is defined, the total potential U of [Equation \(1.6\)](#) is continuously differentiable and the force $\mathbf{F} = -\nabla U$ is continuous, in particular across the influence boundaries $\rho_i = \rho_0$.*

Proof. U_{att} is a polynomial in $\mathbf{p}$ (in the hybrid form, two pieces with equal value and gradient at $d = d^*$). Inside its influence region each $U_{\text{rep},i}$ is a smooth function of ρ_i , which is differentiable where its closest point is unique. At $\rho_i = \rho_0$ the inside expressions [Equations \(1.4\)](#) and [\(1.5\)](#) contain the factor $(1/\rho_i - 1/\rho_0)$ and vanish, which is the value of the outside branch. A sum of C^1 functions is C^1 . $\square$

The second derivative does jump at $\rho_i = \rho_0$: along $\nabla\rho_i$ the inside branch has the curvature $k_{\text{rep}}(3/\rho_i^4 - 2/(\rho_0\rho_i^3))$, equal to k_{rep}/ρ_0^4 at the boundary, while the outside branch has none; [Section 1.7.3](#) uses this curvature.

Theorem 1.7 (Descent, safety and convergence in continuous time). *Let $\mathbf{p}(t)$ solve $\dot{\mathbf{p}} = \mathbf{F}(\mathbf{p}) = -\nabla U(\mathbf{p})$ from a start $\mathbf{p}(0) \in \mathcal{C}_{\text{free}}$ with $U_0 = U(\mathbf{p}(0))$, and assume that ∇U is locally Lipschitz along the trajectory. Then:*

- (i) $U(\mathbf{p}(t))$ is non-increasing, with $\frac{d}{dt}U(\mathbf{p}(t)) = -\|\nabla U(\mathbf{p}(t))\|^2$;
- (ii) the drone never touches an obstacle: $\rho_i(\mathbf{p}(t)) \geq \rho_{\min} > 0$ for all t and i , where $\rho_{\min} = (1/\rho_0 + \sqrt{2U_0/k_{\text{rep}}})^{-1}$;
- (iii) $\mathbf{p}(t)$ stays in a bounded neighbourhood of the goal – within $\sqrt{2U_0/k_{\text{att}}}$ with the quadratic attraction [Equation \(1.2\)](#), within $U_0/(k_{\text{att}}d^*) + d^*/2$ with the hybrid one [Equation \(1.3\)](#) – and converges to the set of critical points of U ; if these are isolated, it converges to one of them.

Proof. (i) By the chain rule, $\frac{d}{dt}U(\mathbf{p}(t)) = \nabla U(\mathbf{p})^\top \dot{\mathbf{p}} = -\|\nabla U(\mathbf{p})\|^2 \leq 0$. (ii) Every term of U is non-negative, so $U \geq U_{\text{rep},i} = \frac{1}{2}k_{\text{rep}}(1/\rho_i - 1/\rho_0)^2$ whenever $\rho_i \leq \rho_0$. Along the trajectory $U \leq U_0$, hence $1/\rho_i - 1/\rho_0 \leq \sqrt{2U_0/k_{\text{rep}}}$, which rearranges to $\rho_i \geq \rho_{\min}$; the bound is trivially true when $\rho_i > \rho_0$. (iii) Likewise $U_{\text{att}}(\mathbf{p}) \leq U \leq U_0$ bounds the trajectory: with the quadratic attraction $\frac{1}{2}k_{\text{att}}d^2 \leq U_0$ gives $d \leq \sqrt{2U_0/k_{\text{att}}}$, and with the hybrid attraction the branch $k_{\text{att}}d^*d - \frac{1}{2}k_{\text{att}}(d^*)^2 \leq U_0$ gives $d \leq U_0/(k_{\text{att}}d^*) + d^*/2$ for $d > d^*$, while $d \leq d^*$ on the other branch. The sublevel set $S = \{\mathbf{p}: U(\mathbf{p}) \leq U_0\}$ is closed, bounded and, by (ii), at

positive distance from every obstacle, so it is a compact subset of $\mathcal{C}_{\text{free}}$ that the trajectory never leaves. LaSalle’s invariance principle (??) then says that $\mathbf{p}(t)$ converges to the largest invariant subset of $\{\mathbf{p} \in S: \nabla U(\mathbf{p}) = \mathbf{0}\}$, the critical points of U in S ; if these are isolated, a continuous trajectory converging to the set converges to one point of it. $\square$

The theorem is the good news and the bad news in one statement: in continuous time the method is *safe*, with an explicit bound $\rho_{\min}$, and the drone always arrives *somewhere* – at a critical point of U , which is the goal only under a condition, and never the only one.

Proposition 1.8 (The goal as the minimum, and when it is not). *Consider U of Equation (1.6) with the quadratic or hybrid attraction.*

- (a) *If $\rho_i(\mathbf{p}_g) \geq \rho_0$ for every obstacle, then $U(\mathbf{p}_g) = 0 \leq U(\mathbf{p})$ for all $\mathbf{p}$: the goal is the global minimum of U and a critical point.*
- (b) *If $\rho_i(\mathbf{p}_g) < \rho_0$ for some obstacle and the repulsions do not cancel exactly at the goal, then $\mathbf{F}(\mathbf{p}_g) = \sum_i \mathbf{F}_{\text{rep},i}(\mathbf{p}_g) \neq \mathbf{0}$: the goal is not a critical point, and no trajectory of Theorem 1.7 ends there.*
- (c) *If every repulsive term is multiplied by d^n with $d = \|\mathbf{p} - \mathbf{p}_g\|$ and $n \geq 1$, the goal is again the global minimum, and for $n \geq 2$ it is a critical point.*

Proof. (a) All terms are non-negative and vanish at the goal, which lies outside every influence region; a global minimum of a C^1 function is critical. (b) $\mathbf{F}_{\text{att}}(\mathbf{p}_g) = \mathbf{0}$ by Equation (1.2), and the repulsive terms with $\rho_i < \rho_0$ are non-zero by Equation (1.5). (c) The modified terms $U_{\text{rep},i} d^n$ are non-negative and vanish at $d = 0$, so $U(\mathbf{p}_g) = 0$ is the global minimum. Their gradient is $d^n \nabla U_{\text{rep},i} + n d^{n-1} U_{\text{rep},i} \nabla d$ (Section 1.7.2), and both summands contain a positive power of d when $n \geq 2$. $\square$

Remark 1.9 (Critical points other than the goal cannot be avoided). Koditschek and Rimon [7] showed, using Morse theory, that on a sphere world (a disc with M disjoint disc obstacles removed) every smooth potential that has the goal as its only minimum and is maximal on the obstacle boundaries has at least M other critical points. The reason is topological: the free space has Euler characteristic $1 - M$, and the count “minima minus saddles plus maxima” of a Morse function equals it, so one minimum and no interior maxima force M saddles. The best one can hope for is that all of them are *saddles*, from which almost every start escapes, rather than *minima*, which capture a whole basin. Khatib’s field does not achieve even that (Section 1.8.1); the navigation functions of Section 1.8 do.

1.7 The failure modes

Koren and Borenstein [8] listed four problems that they had met on their own robots: traps due to local minima, no passage between closely spaced obstacles, oscillations in the presence of obstacles, and oscillations in narrow passages. Each subsection below takes one failure, produces it with a function of `ch15_potential_fields.py` (the self-test asserts that the failure occurs), shows it in a figure, and gives the formula that explains it. Every number is printed by the self-test.

1.7.1 Local minima: the obstacle between the drone and the goal

Put the drone at $(0, 4)$ in the scene of Example 1.5, so that the disc sits exactly between it and the goal. By symmetry the force on the axis $y = 4$ is horizontal, and for $1 \leq x < 3$, inside the influence region and left of the disc, its x -component is

$$F_x(x) = (8 - x) - \left(\frac{1}{3 - x} - \frac{1}{2} \right) \frac{1}{(3 - x)^2}, \quad (1.7)$$

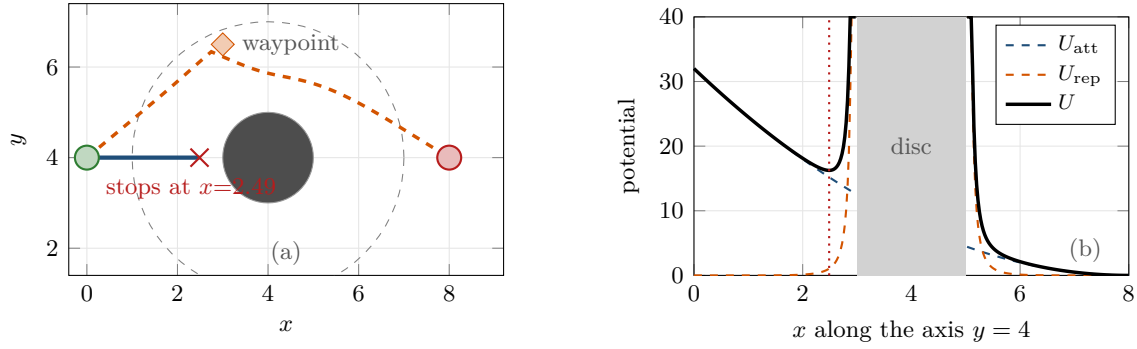


Figure 1.4. Failure mode 1: local minimum. (a) From $(0, 4)$ the drone stops at $x = 2.49$, where the attraction $(8 - x)$ and the repulsion cancel; the dashed orange path is the same drone steered through a waypoint at $(3, 6.5)$ from a global planner (Section 1.8). (b) The potentials along the axis $y = 4$: attractive bowl (blue), repulsive chimney (orange) and their sum (black), with a dip at $x = 2.49$ (dotted red line).

because the clearance is $\rho = 3 - x$ and $\nabla\rho = (-1, 0)$. At $x = 1$ the repulsion is zero and $F_x = 7$; as $x \rightarrow 3$ the repulsion tends to $-\infty$. F_x is continuous, so it has a root, and bisection puts it at $x^* = 2.4872$. The run `local_minimum_case()` confirms it: the drone flies along the axis, slows down, and is declared stuck at $x = 2.4872$ after 354 steps (Figure 1.4a). Along the axis the potential has a genuine dip at x^* (Figure 1.4b): U falls from 32 at the start to about 16 at x^* and then climbs the chimney. This is the **classic local minimum**: the attraction says “forward”, the repulsion says “back”, and where they cancel the drone stops, although a free path over or under the disc exists and is only 12 % longer than the straight line.

The dip is a minimum *along the axis* only. The Hessian of U at $(x^*, 4)$, computed by central differences of the analytic force, has the eigenvalues 36.96 in the x -direction and -2.64 in the y -direction: the point is a **saddle**, and a drone slightly off the axis is pushed further off by the repulsion faster than the attraction pulls it back. For a single convex obstacle the unavoidable critical point of Remark 1.9 is a saddle and only the starts exactly on the axis are trapped – little comfort in practice, because a drone near the axis lingers next to the saddle, and any asymmetry (a second obstacle, a wall, a non-convex shape) turns the saddle into a true minimum. The basin experiment of Section 1.8.1 uses two overlapping discs, and there 41 % of the starts end in a genuine local minimum with two positive Hessian eigenvalues.

1.7.2 Goals non-reachable with obstacles nearby (GNRON)

Move the goal to $(5.4, 4)$, 0.4 units from the right edge of the disc and therefore well inside its influence region, and start the drone at $(8, 6)$. The run `gnron_case()` stops at $(5.968, 4.001)$, 0.568 units short of the goal, and stays there (Figure 1.5). The cause is Proposition 1.8(b): at the goal the attraction vanishes but the repulsion does not; with $\rho = 0.4$ it has the magnitude $(1/0.4 - 1/2)/0.4^2 = 12.5$ and points away from the disc. The minimum of U is where the pull towards the goal balances the push away from the disc, $k_{\text{att}} d = k_{\text{rep}}(1/\rho - 1/\rho_0)/\rho^2$ with $\rho = 0.4 + d$, whose solution $d = 0.568$ is exactly where the drone stops. Ge and Cui [5] named the phenomenon **GNRON**, *goals non-reachable with obstacles nearby*; a landing pad next to a building or a formation slot beside a neighbour is enough to produce it.

The standard fix multiplies each repulsive term by a power of the distance to the goal, so that the repulsion fades out exactly where the goal is:

$$U'_{\text{rep},i}(\mathbf{p}) = U_{\text{rep},i}(\mathbf{p}) d(\mathbf{p})^n, \quad d(\mathbf{p}) = \|\mathbf{p} - \mathbf{p}_g\|, \quad n \geq 1. \quad (1.8)$$

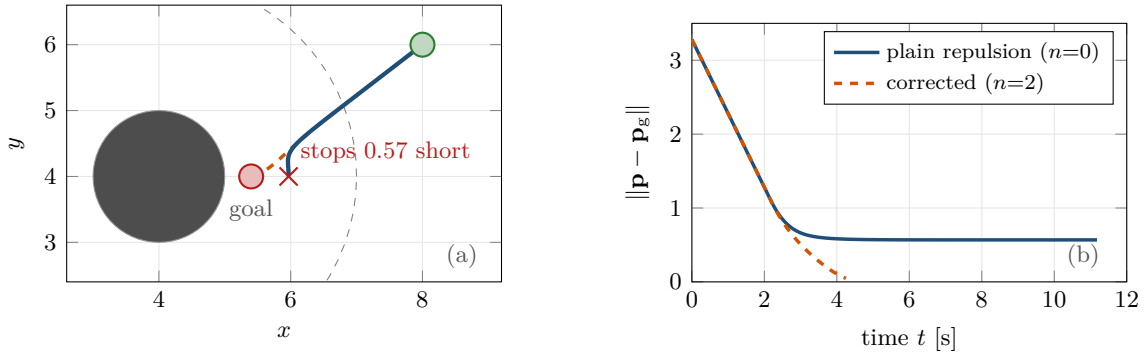


Figure 1.5. Failure mode 2: the goal lies 0.4 from the disc, inside the influence region. (a) With Khatib’s repulsion (solid blue) the drone from (8, 6) stops 0.57 short of the goal, where attraction and repulsion balance; with the repulsion multiplied by d^2 (dashed orange) it arrives. (b) Distance to the goal against time: the plain field levels off at 0.57, the corrected one reaches the tolerance 0.05 after 4.25 time units.

The product rule with $\nabla d = (\mathbf{p} - \mathbf{p}_g)/d$ gives the corrected force, line 12 of Algorithm 1.1:

$$\mathbf{F}'_{\text{rep},i} = -\nabla(U_{\text{rep},i} d^n) = d^n \mathbf{F}_{\text{rep},i} - \frac{n}{2} k_{\text{rep}} \left(\frac{1}{\rho_i} - \frac{1}{\rho_0} \right)^2 d^{n-1} \frac{\mathbf{p} - \mathbf{p}_g}{d}. \quad (1.9)$$

The first term is the old repulsion, scaled down near the goal; the second is a new pull *towards* the goal, present only inside influence regions. By Proposition 1.8(c) the goal is now the global minimum, and for $n \geq 2$ also a critical point; with $n = 1$ the second term keeps a non-zero magnitude at the goal whose direction depends on the side of approach, a conic pull that makes the drone chatter with the amplitude $\Delta t v_{\text{max}}$ of one step. Here that is 0.01, well below the goal tolerance $\varepsilon_g = 0.05$, so the run is declared reached and the chatter appears only when the tolerance is tightened below it (Exercise 1.4); this is why Ge and Cui recommend $n = 2$, which is the default of the code. With $n = 2$ the run of Figure 1.5 reaches the goal after 425 steps. Two warnings: the factor changes the field everywhere (d^2 is 100 ten units away, so k_{rep} must be retuned for the size of the scene), and it cures only this failure – from (0, 4) the corrected field still stops the drone on the axis, at a different x .

Waypoints next to walls

The most common way to meet GNRON in a drone system is not a goal next to a building but a *waypoint* next to one: a grid planner (??) happily routes through cells that touch obstacles, and a potential field asked to reach such a waypoint with a tolerance smaller than the offset of Proposition 1.8(b) hovers in front of it for ever. Either inflate the obstacles by more than the drone radius before planning, so that waypoints stay outside ρ_0 , or accept a waypoint as reached at a tolerance of the order of ρ_0 , or use Equation (1.9) for the last leg.

1.7.3 Oscillation in narrow passages and with discrete time steps

Two discs of radius 1 centred at (4, 2.5) and (4, 5.5) leave a passage of width 1.0 on the axis $y = 4$; the drone starts at (0, 4.3) and its goal is (8, 4.2), slightly off the axis as in any real scene. Figure 1.6 shows three runs of `corridor_case()` through this passage. With $\Delta t = 0.01$ and $v_{\text{max}} = 1$ the drone glides through, reverses its lateral direction once, never deviates more than 0.161 from the axis and keeps a clearance of 0.498; with $\Delta t = 0.05$ and no speed limit it zigzags (8 reversals inside the passage, amplitude 0.410, clearance 0.342); with $\Delta t = 0.10$ and

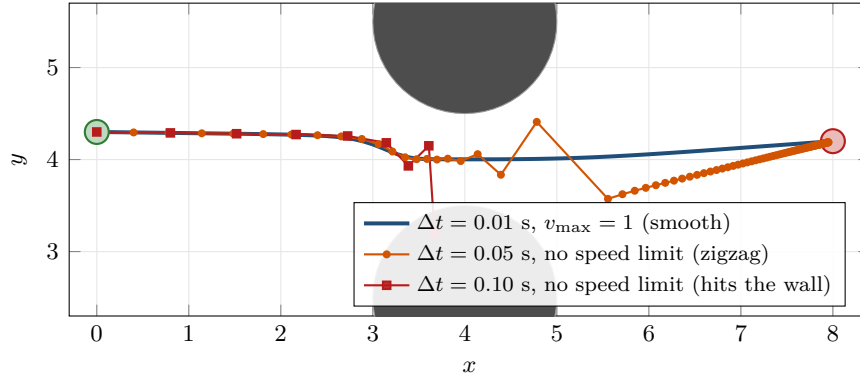


Figure 1.6. Failure mode 3: oscillation in a passage of width 1.0, crossed with three control periods. With $\Delta t = 0.01$ and the speed limit (blue) the drone glides through; with $\Delta t = 0.05$ and no limit (orange, one mark per step) it zigzags between the walls; with $\Delta t = 0.10$ and no limit (red) the second step inside the passage overshoots the axis by more than the half-width and the drone crashes. The lateral stiffness at the midpoint is 81, so $\Delta t \lambda_{\max}$ is 0.81, 4.05 and 8.1.

no speed limit it hits the lower wall after 8 steps. The field is the same in all three runs; what differs is the step.

Proposition 1.10 (Discrete-time stability near a minimum). *Let $\mathbf{p}^*$ be a strict local minimum of a C^2 potential U , with Hessian $\mathbf{H} = \nabla^2 U(\mathbf{p}^*)$ and eigenvalues $0 < \lambda_1 \leq \dots \leq \lambda_{\max}$. The update $\mathbf{p}_{k+1} = \mathbf{p}_k - \Delta t \nabla U(\mathbf{p}_k)$ (line 12 of Algorithm 1.2 while the speed limit is inactive) is locally asymptotically stable at $\mathbf{p}^*$ if $\Delta t \lambda_{\max} < 2$ and unstable if $\Delta t \lambda_{\max} > 2$; at $\Delta t \lambda_{\max} = 2$ the linearisation is marginally stable and decides nothing. Along the eigenvector of λ_i the error is multiplied by $1 - \Delta t \lambda_i$ per step: it shrinks monotonically for $\Delta t \lambda_i < 1$, alternates in sign for $1 < \Delta t \lambda_i < 2$, and grows for $\Delta t \lambda_i > 2$.*

Proof. Write $\mathbf{e}_k = \mathbf{p}_k - \mathbf{p}^*$. Since $\nabla U(\mathbf{p}^*) = \mathbf{0}$, Taylor's theorem gives $\mathbf{e}_{k+1} = (\mathbf{I} - \Delta t \mathbf{H})\mathbf{e}_k + o(\|\mathbf{e}_k\|)$. The symmetric matrix $\mathbf{I} - \Delta t \mathbf{H}$ has the eigenvalues $1 - \Delta t \lambda_i$, and the linear recursion is asymptotically stable when all of them have modulus below 1, that is, when $\Delta t \lambda_i < 2$ for all i , and unstable when one of them has modulus above 1; the nonlinear system inherits both properties from its linearisation (Lyapunov's indirect method, ??), which says nothing when a modulus equals 1. The statements about monotone and alternating decay are the sign and size of the factor. $\square$

In a passage the potential is stiff in the lateral direction. At the midpoint of a passage of width w between two walls at clearance $\rho = w/2$, differentiating Equation (1.4) twice gives the lateral curvature

$$\lambda_y = k_{\text{att}} + 2k_{\text{rep}} \left(\frac{3}{\rho^4} - \frac{2}{\rho_0 \rho^3} \right), \quad \rho = \frac{w}{2}, \quad (1.10)$$

the sum of the bowl and of the two walls. For $w = 1$ this is $1 + 2(48 - 8) = 81$, the **stiffness** that the code computes numerically as the largest Hessian eigenvalue, and Proposition 1.10 explains the three runs: $\Delta t \lambda_y = 0.81 < 1$ decays monotonically, $4.05 > 2$ diverges, and 8.1 diverges so fast that the second step leaves the passage. The stiffness grows roughly like the inverse fourth power of the width, 204 for $w = 0.8$ and 668 for $w = 0.6$: a step that is safe in open space is unstable in every passage narrower than some width, and no single Δt serves all scenes.

Table 1.3 repeats the experiment with the speed limit switched on. The limit is a strong medicine: with $v_{\max} = 1$ the run with $\Delta t = 0.05$ and $w = 1$ crosses with a single reversal although its stability number is 4.05, because the clipped step is at most $\Delta t v_{\max} = 0.05$ long and cannot overshoot the axis by more than that; what remains of the instability is chatter of that amplitude (the 51 reversals of the blocked run with $w = 0.8$). Koren and Borenstein's

Table 1.3. The passage experiment of `corridor_case()` with the speed limit $v_{\max} = 1$ active: width w , control period Δt , outcome, steps, lateral reversals inside the passage, largest lateral excursion, stiffness Equation (1.10) and the stability number $\Delta t \lambda_y$. Widths 0.6 and 0.8 are never entered (Section 1.7.4); with $\Delta t = 0.05$ the blocked drone chatters in front of the entrance.

w	Δt	Outcome	Steps	Reversals	Excursion	λ_y	$\Delta t \lambda_y$
0.6	0.01	stuck	446	0	0.065	667.7	6.68
0.6	0.05	stuck	88	24	0.067	667.7	33.4
0.8	0.01	stuck	619	0	0.122	204.1	2.04
0.8	0.05	stuck	120	51	0.117	204.1	10.2
1.0	0.01	reached	1003	1	0.161	81.0	0.81
1.0	0.05	reached	200	1	0.155	81.0	4.05
1.2	0.01	reached	1002	1	0.190	38.0	0.38
1.2	0.05	reached	200	1	0.183	38.0	1.90
1.6	0.01	reached	1001	1	0.221	11.7	0.12
1.6	0.05	reached	199	1	0.218	11.7	0.59

“oscillations in the presence of obstacles” also arise in continuous time when the drone has inertia: the second-order model $m\ddot{\mathbf{p}} = \mathbf{F} - b\dot{\mathbf{p}}$ oscillates unless the damping b is large compared with $\sqrt{m\lambda}$, a velocity controller with latency behaves like a lightly damped mass, and noisy distances produce a jittering force.

1.7.4 No passage between closely spaced obstacles

Narrow the passage to $w = 0.6$ (discs centred at $(4, 2.7)$ and $(4, 5.3)$). The axis stays 0.3 clear of both discs, so the passage is free, and a drone that starts inside it is carried through: on the axis at $x = 4$ the factor $4 - x$ of Equation (1.11) below kills the horizontal repulsion, so $F_x(4) = 8 - 4 = 4 > 0$ points at the goal, and the run `simulate((4,4), (8,4.2), corridor(0.6), ApfParams())` reaches it in 599 steps with a minimum clearance of 0.300. Yet the drone from $(0, 4.3)$ never gets in: it stops at $(3.163, 4.005)$, in front of the entrance, after 446 steps (Figure 1.7a). On the axis the vertical components of the two repulsions cancel and their horizontal components add. With $r(x) = \sqrt{(4-x)^2 + 1.3^2}$ the distance to either centre and $\rho = r - 1$,

$$F_x(x) = (8 - x) - 2k_{\text{rep}} \left(\frac{1}{\rho} - \frac{1}{\rho_0} \right) \frac{1}{\rho^2} \frac{4 - x}{r}, \quad (1.11)$$

plotted in Figure 1.7b. It is positive far away, dips to -6.01 just before the entrance, has its first root at $x = 3.163$ (a second, unstable root sits near $x = 3.91$), and is positive again inside the passage. The drone stops at the first root: two walls that each say “keep away” add up to “go back” in front of the gap, although neither wall is in the way. Table 1.3 shows that $w = 0.8$ is blocked as well. Raising k_{att} or lowering k_{rep} or ρ_0 opens the gap at the price of a smaller clearance everywhere else (Exercise 1.6); no setting passes every doorway and keeps every margin, which is what makes potential fields unsuitable as the *only* planner in cluttered spaces.

1.7.5 The Koren–Borenstein critique, summarised honestly

Koren and Borenstein [8] tested their own implementation, the *virtual force field*, on a mobile robot with sonar sensors, the repulsion being summed over the occupied cells of a certainty grid. They found the four problems above, concluded that potential fields are unsuitable for real-time obstacle avoidance, and replaced them with the **vector field histogram** (VFH) of

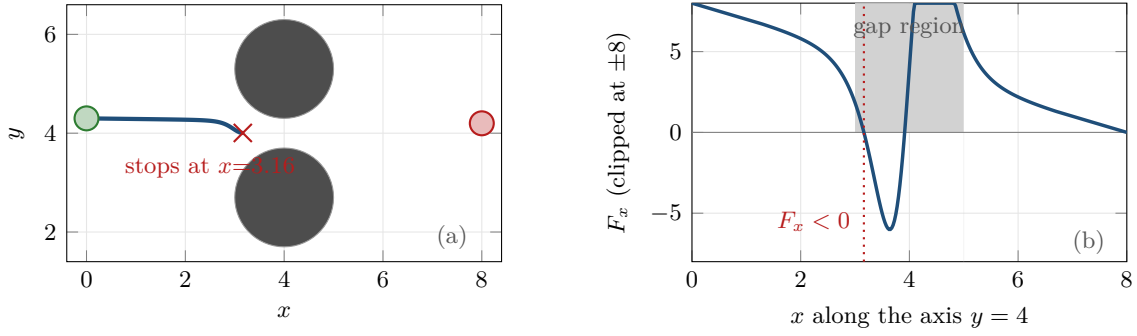


Figure 1.7. Failure mode 4: a free passage of width 0.6 that the drone does not enter. (a) The trajectory from (0, 4.3) ends at $x = 3.16$, in front of the gap. (b) The horizontal force Equation (1.11) along the axis, clipped at ± 8 : the two repulsions add up in front of the entrance and push the drone back before it reaches the point where they cancel.

Borenstein and Koren [2], which picks a steering direction from a polar histogram instead of adding vectors. Three decades later the honest summary is this.

- *Traps and blocked passages are structural.* They follow from the algebra of Equations (1.7) and (1.11), and Remark 1.9 shows that critical points other than the goal cannot be avoided by any potential; gains and influence distances move the failures around, they do not remove them.
- *The oscillations are partly implementation-dependent.* The first-order controller with exact distances never oscillates in continuous time (Theorem 1.7); what oscillated in their experiments was discretisation, robot dynamics and sonar noise. But the stiffness Equation (1.10) leaves every discrete controller a passage too narrow for it.
- *The verdict was about a role, not about the idea.* It concerned the potential field as the *whole* obstacle-avoidance layer of a robot driven by raw sonar readings, not potential terms inside a larger system (Section 1.11) or navigation functions.

1.8 Remedies

Navigation functions. Rimon and Koditschek [7, 12] asked for the best a potential can be: a **navigation function** is a smooth potential on a bounded free space that is maximal on the obstacle boundaries, has the goal as its unique minimum, and whose other critical points are non-degenerate saddles. By Theorem 1.7 and Remark 1.9 this is the strongest possible guarantee: every start outside a set of measure zero reaches the goal, and no start collides. For a **sphere world** – a disc of radius r_0 centred at $\mathbf{c}_0$ with M disjoint disc obstacles $(\mathbf{c}_i, r_i)$ inside – they construct one explicitly:

$$\varphi_\kappa(\mathbf{p}) = \frac{\|\mathbf{p} - \mathbf{p}_g\|^2}{(\|\mathbf{p} - \mathbf{p}_g\|^{2\kappa} + \beta(\mathbf{p}))^{1/\kappa}}, \quad \beta(\mathbf{p}) = (r_0^2 - \|\mathbf{p} - \mathbf{c}_0\|^2) \prod_{i=1}^M (\|\mathbf{p} - \mathbf{c}_i\|^2 - r_i^2), \quad (1.12)$$

and prove that φ_κ is a navigation function for every sufficiently large κ : β is positive in the free space and zero on every boundary, so φ_κ is 0 at the goal and 1 on the obstacles, and the large exponent sharpens the goal term until the obstacle terms can only produce saddles. The result matters for what it says: local minima are not a law of nature but a defect of the sum in Equation (1.6). It is rarely used on drones, because it needs the complete obstacle geometry in closed form, a large κ makes the field extremely flat far from the goal and extremely steep near it, and moving obstacles break the construction.

Random walks and simulated annealing. Barraquand and Latombe [1] accepted the local minima and added an escape: when the descent stalls, execute a **random walk** of random length, then descend again, and repeat until the goal is reached. The escape used by `simulate_many()` is the simplest version, a straight move of random direction and random length between 0.5 and 2 units at full speed, abandoned early if it would come within 0.1 of an obstacle. **Simulated annealing** is the principled form of the same idea: accept an uphill move with probability $\exp(-\Delta U/T)$ and lower the temperature T over time. Both trade determinism and path quality for a probabilistic guarantee, and Section 1.8.1 measures what the trade buys.

Harmonic potentials. Connolly, Burns and Weiss [4] proposed to compute U as a solution of Laplace’s equation, $\nabla^2 U = 0$ in the free space, with $U = 1$ on the obstacle boundaries and $U = 0$ at the goal. A **harmonic potential** has no interior extrema by the maximum principle, so it has no local minima, and the descent reaches the goal from every start. The price is a global computation: Laplace’s equation must be solved numerically on a grid covering the workspace and re-solved whenever the map changes. Its discrete cousin, the wavefront planner of Choset et al. [3], assigns every free cell its number of steps to the goal by a breadth-first sweep – Dijkstra’s algorithm of ?? under another name. A potential without local minima costs a global search.

A potential field as the local layer under a global plan. The remedy that matters for this book is to stop asking the field to do global planning. A global planner – A^* on a grid (??), D^* Lite when the map changes (??), CBS for the swarm (??) – delivers waypoints along a path that is known to exist, and the field descends to each waypoint in turn. Each leg is short, its goal is close and in view, and the local minima of the field with respect to the far-away goal are simply not there. The dashed path of Figure 1.4a is the local-minimum case of Section 1.7.1 steered through the single waypoint (3,6.5): the drone reaches the waypoint, then the goal, in 1 188 steps, without any change to the field; `follow_waypoints()` in `ch15_potential_fields.py` is the whole mechanism, a loop that runs Algorithm 1.2 towards each waypoint with a loose tolerance and aborts when a leg does not end with “reached”. The field keeps its job as the reactive layer – obstacles that were not on the map, and other drones, still repel – and two rules keep the arrangement honest: waypoints must be reached with a tolerance of the order of ρ_0 , or moved away from walls, because of GNRON; and a stalled leg (line 8) must trigger a replanning of the global path, which is the loop of ??.

1.8.1 A generated experiment: the basin of a local minimum

How much of the start space does a local minimum capture, and how much does a random walk recover? `basin_experiment()` answers this for a scene with two obstacles: discs of radius 1 centred at (4,3.2) and (4,5.0), which overlap into a single peanut-shaped obstacle with a concave waist, and the goal at (8,4.1). Drones are released from 441 starts on a 21×21 grid covering $x \in [0, 2]$, $y \in [0.5, 7.7]$, and Algorithm 1.2 runs from all of them at once (Figure 1.8a). With the plain field, 262 starts reach the goal (59.4%) and 179 are declared stuck (40.6%); none collides. All 179 end at the *same* point, (2.676,4.1), in front of the waist. Unlike the saddle of Section 1.7.1, this is a true minimum: the two repulsions form a valley along the symmetry axis, and the Hessian of U there has the eigenvalues 9.87 and 24.89, both positive. Its basin is the band of starts within about 1.4 of the axis, narrowing to about 1.1 for starts already inside the influence region.

With the random-walk escape switched on, 386 starts reach the goal (87.5%), none is left stuck, and 55 run into the horizon of 4 000 steps while still walking and descending; the drones that do arrive need on average 1.24 escapes and 1 428 steps, against 1 018 for the plain runs

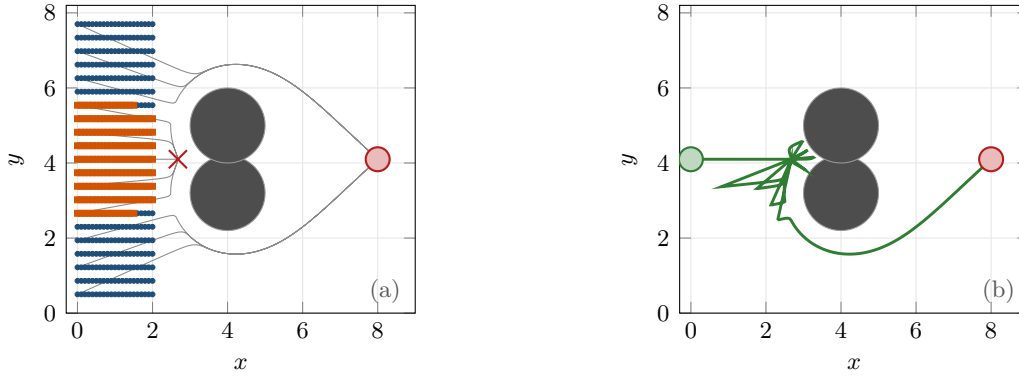


Figure 1.8. The basin experiment. (a) 441 starts in front of a peanut-shaped obstacle with the goal at (8, 4.1): blue dots reach the goal, orange squares end in the local minimum at (2.676, 4.1) (red cross), grey curves are the trajectories from the column $x = 0$; the basin is a band around the symmetry axis holding 41 % of the starts. (b) A single-drone run from the trapped start (0, 4.1) with random-walk escapes (green), drawn from its own random stream: after 8 walks and 3810 steps the drone finds the way round the upper lobe. In the batch run of (a) the same start is still walking when the horizon expires.

(averaged over all 441 starts the escape count is 2.48, because the 55 drones that never arrive keep walking). Figure 1.8b shows a separate single-drone run from the trapped start (0, 4.1) on the axis, with its own random stream: eight random walks, 3810 steps, and a final route over the upper lobe. In the batch run that start is still walking when the horizon expires, because the outcome of a random escape is a random variable and not a property of the start. The random walk converts a certain failure into a probable, slow success with an unpredictable path – a crutch worth having in a local layer, and no substitute for a global plan.

The failures of this chapter are properties of *regions* of the start space, not of the scene alone, and nothing in a run signals them except the stuck detector: a single test from a start outside the band would have passed. Sweep a grid of starts as `basin_experiment()` does, report the fractions that arrive and that stall, and keep the detector of Algorithm 1.2 in the deployed code, because it is the only component that can tell the rest of the system that the field has failed; ?? turns this into an experiment protocol.

1.9 Swarms: inter-agent repulsion, flocking and consensus

Nothing in Algorithm 1.1 requires an obstacle to be static. The other drones of the swarm are obstacles too, discs of radius r that move, and the natural extension is a pairwise repulsion with the clearance $\rho_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\| - 2r$ between drones i and j :

$$\mathbf{F}_i = \mathbf{F}_{\text{att},i} + \sum_k \mathbf{F}_{\text{rep},k}(\mathbf{p}_i) + \sum_{j \neq i, \rho_{ij} < \rho_a} k_a \left(\frac{1}{\rho_{ij}} - \frac{1}{\rho_a} \right) \frac{1}{\rho_{ij}^2} \frac{\mathbf{p}_i - \mathbf{p}_j}{\|\mathbf{p}_i - \mathbf{p}_j\|}, \quad (1.13)$$

with its own gain k_a and range ρ_a . This **inter-agent repulsion** is symmetric – the term that j exerts on i is the negative of the term i exerts on j – so the pair shares one potential. `simulate_swarm()` runs it for six drones on a ring of radius 3 whose goals are the opposite points of the ring, so that all six nominal paths cross at the centre at the same moment. Without the pair term the minimum separation over the run is 0.000: all six meet. With $k_a = 1$, $\rho_a = 1.5$ and $r = 0.25$ it is 1.044, twice the touching distance $2r = 0.5$, and after 30 time units every drone is within 0.3 of its goal (Figure 1.9b). The symmetric scene is also the swarm version of the local minimum: two drones exactly head-on would stop facing each other,

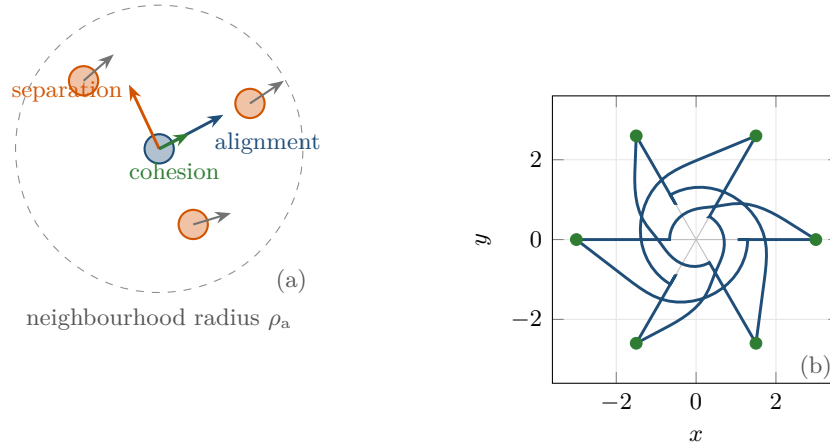


Figure 1.9. Inter-agent repulsion and flocking. (a) Reynolds's three rules seen from the focal drone (blue): separation pushes it away from close neighbours, alignment turns its velocity towards the neighbours' average heading (grey arrows), cohesion pulls it towards their centroid; all three use only the neighbours within the radius ρ_a . (b) Six drones on a ring of radius 3 swap places with their opposite points. With pairwise repulsion (blue) the closest approach is 1.044; the grey lines are the paths without it, which all pass through the centre at the same time.

since their attractions and repulsions cancel. Symmetric repulsion has no notion of who yields; the reciprocal velocity obstacles of ?? exist precisely to supply one.

Flocking and consensus. Reynolds [11] animated flocks with three local rules that every agent applies to the neighbours within a radius (Figure 1.9a): **separation**, steer away from neighbours that are too close; **alignment**, steer towards the average velocity of the neighbours; and **cohesion**, steer towards their centroid. The three rules are potential fields and one thing more. Separation is Equation (1.13). Cohesion is an attractive potential whose goal moves, $\frac{1}{2}k_c \|\mathbf{p}_i - \bar{\mathbf{p}}\|^2$ with $\bar{\mathbf{p}}$ the centroid of the neighbours. Alignment is not the gradient of any potential over positions; it acts on velocities. Write $N_i = \{j \neq i: \|\mathbf{p}_i - \mathbf{p}_j\| \leq \rho_a\}$ for the neighbours of drone i within the interaction radius ρ_a , and let $\alpha > 0$ be the alignment gain, small enough that $\alpha |N_i| < 1$ for every drone, so that each update is a convex combination of the velocities:

$$\mathbf{v}_i \leftarrow \mathbf{v}_i + \alpha \sum_{j \in N_i} (\mathbf{v}_j - \mathbf{v}_i), \quad (1.14)$$

and Equation (1.14) is exactly the consensus update of ?? applied to the velocities: on a connected neighbourhood graph all velocities converge to a common value and the flock moves as one. Olfati-Saber [10] made the correspondence precise and proved that a pairwise potential (separation and cohesion in one smooth term), a velocity-consensus term and a navigation term together produce stable flocking without collisions. Formation keeping is the same construction with a prescribed geometry: a spring of rest length d_{ij} between designated pairs, $\frac{1}{2}k_s(\|\mathbf{p}_i - \mathbf{p}_j\| - d_{ij})^2$, is the displacement-based formation controller of ?? read as a potential (`simulate_swarm()` offers such springs through its `k_spring` argument), and ?? supplies the convergence proofs that this chapter, with its local minima, cannot.

1.10 Implementation notes

`ch15_potential_fields.py` implements everything in this chapter for a point drone in the plane with disc obstacles. Positions are NumPy arrays of shape $(2,)$ or $(N,2)$, and every function broadcasts over the leading axis, so that `simulate_many()` advances N drones with

one call per step and the basin experiment moves its 441 drones for 4000 steps in about a second. [Listing 1.1](#) is the repulsive force, a direct transcription of [Equations \(1.5\) and \(1.9\)](#); the self-test checks the analytic forces against central differences of the potential at 200 random points for five parameter settings.

Listing 1.1. The repulsive force (from `code/ch15_potential_fields.py`). `prm` holds the gains; `obs.distance(p)` returns the clearance ρ and $\nabla\rho$; `prm.n_gnron` is the exponent n of [Equation \(1.9\)](#), 0 for Khatib’s original; `EPS` is 10^{-12} .

```

1 def repulsive_force(p, goal, obstacles, prm):
2     """F_rep = -grad U_rep, summed over the obstacles within rho0.
3
4     Plain (n = 0): k_rep (1/rho - 1/rho0) / rho^2 * grad rho.
5     GNRON (n >= 1): the same times d^n, minus
6                     (n/2) k_rep (1/rho - 1/rho0)^2 d^(n-1) * grad d,
7     where d = ||p - goal|| and grad d = (p - goal)/d points away from the goal.
8     """
9     diff, d = _goal_offset(p, goal)
10    n = prm.n_gnron
11    f = np.zeros_like(diff)
12    for obs in obstacles:
13        rho, grad_rho = obs.distance(p)
14        rho_c = np.maximum(rho, EPS)
15        gap = 1.0 / rho_c - 1.0 / prm.rho0
16        term = prm.k_rep * gap / rho_c ** 2 * grad_rho
17        if n > 0:
18            grad_d = diff / np.maximum(d, EPS)
19            term = term * d ** n - 0.5 * n * prm.k_rep * gap ** 2 * d ** (n -
20    1) * grad_d
21        f = f + np.where(rho < prm.rho0, term, 0.0)
22    return f

```

Step size. [Proposition 1.10](#) gives the rule: $\Delta t \lambda_{\max} < 1$ at the stiffest point the drone is meant to visit, with $\lambda_{\max}$ from [Equation \(1.10\)](#) for the narrowest passage in the scene; `stiffness()` computes the largest Hessian eigenvalue at a point by differencing the analytic force. A second rule protects against the crash of [Figure 1.6](#): with the speed limit active a step moves the drone by at most $\Delta t v_{\max}$, so keeping that product below the smallest clearance you accept rules out the crash, though not the chatter.

Force clipping. Clip the *total* force, as line 11 does, not its terms: scaling the vector keeps the direction that the field prescribes, while clipping components or individual repulsions changes the direction and can steer the drone into the very obstacle it is avoiding. The magnitude $k_{\text{rep}}(1/\rho - 1/\rho_0)/\rho^2$ overflows as $\rho \rightarrow 0$; [Listing 1.1](#) floors ρ at `EPS`, the controller reports a collision when $\rho \leq 0$, and a drone already inside an obstacle (a map error, a moving intruder) must be handled explicitly, because the formula changes sign there.

Distance queries. Line 8 of [Algorithm 1.1](#) is where the implementation meets the map. For discs it is a subtraction; for a polygon, the minimum over its edges of the point-to-segment distance of ??; for a grid map, a Euclidean distance transform computed once, with the gradient read off by central differences. With many obstacles, keep them in a uniform grid of buckets or a KD-tree (??) and query only the ones within ρ_0 . Where two obstacles are equally close, $\nabla\rho$ of the obstacle set is undefined and a drone feels the sum of both repulsions, which is the mechanism of [Section 1.7.4](#).

Discrete-time instability and the controller loop. The loop of `simulate_many()` is Algorithm 1.2 with the three tests, the clipped step and the optional random-walk escape of Section 1.8, vectorised over the drones with boolean masks. The stuck detector compares the current position with the position W steps ago; W should cover several periods of any chatter, and the tolerance δ should exceed the chatter amplitude $\Delta t v_{\max}$, otherwise a chattering drone is never declared stuck. A random walk is committed for a whole leg of steps, so that the descent does not cancel it at the next step.

A force is not a velocity

The formula $\mathbf{v} = \mathbf{F}$ hides two unit conversions that a real drone does not forgive. First, the magnitude of $\mathbf{F}$ is unbounded, far from the goal and close to an obstacle, and a velocity controller that receives such a command saturates its motors, loses the direction information and, in the discrete-time model, takes the crash step of Figure 1.6; the speed limit of line 11 is a necessity, not a refinement. Second, a drone has inertia and its velocity loop has latency, so the effective model is second order and lightly damped, and Proposition 1.10 underestimates the oscillation. Either low-pass filter the command, $\mathbf{v}_{k+1} = (1 - \beta)\mathbf{v}_k + \beta \mathbf{F}$ with $\beta < 1$, or hand the force to a controller that knows the dynamics: the dynamic window of ?? or the model predictive controller of ??.

1.11 Where this fits in the drone system

In the drone system

A potential field is the cheapest reactive layer there is: one distance query per nearby obstacle or drone, one vector sum, no search. In the hybrid architecture of ?? it is nevertheless *not* the safety layer. It reacts to positions, not velocities: an intruder approaching head-on is repelled only once it is within ρ_0 , whereas the velocity obstacles of ?? and ORCA (??) reason about relative velocity and time to collision and yield earlier and reciprocally; it has no notion of the drone's dynamics, whereas the dynamic window approach of ?? evaluates only commands the drone can execute; and it has the four failure modes of Section 1.7, which no tuning removes from every scene.

The book therefore keeps potential fields for what they do well: the inter-agent repulsion of Equation (1.13) is the *soft spacing* term that runs at every control cycle between swarm members on top of the time-indexed paths of CBS or ECBS (????), the spring terms of Section 1.9 are the *formation-keeping* controller of ??, and the same quadratic penalties reappear as soft constraints in the model predictive controller of ?. The waypoint-following arrangement of Section 1.8 is the pattern every local layer of ?? follows, with a stalled leg triggering replanning (????). Week 7 of the study plan (??) covers this chapter with ??; its coding exercise is Exercise 1.9.

1.12 Summary

Chapter summary

- A potential field adds a bowl at the goal, $U_{\text{att}} = \frac{1}{2}k_{\text{att}} \|\mathbf{p} - \mathbf{p}_g\|^2$ (conic beyond d^* in the hybrid form), to a wall around every obstacle, $U_{\text{rep}} = \frac{1}{2}k_{\text{rep}}(1/\rho - 1/\rho_0)^2$ for $\rho \leq \rho_0$, and commands the negative gradient of the sum, Equations (1.2) and (1.5).
- The controller is explicit Euler with a speed limit and a stuck detector. In continuous

time the potential decreases monotonically, the drone never touches an obstacle, and it converges to a critical point of U , which is the goal only when nothing cancels the attraction first.

- The four failure modes have formulas: a local minimum where $\mathbf{F}_{\text{att}} + \mathbf{F}_{\text{rep}} = \mathbf{0}$; GNRRON, a goal inside ρ_0 where $\mathbf{F}_{\text{rep}} \neq \mathbf{0}$, cured by the factor d^n ; oscillation once $\Delta t \lambda_{\text{max}} > 1$ and divergence once $\Delta t \lambda_{\text{max}} \geq 2$, with a stiffness growing like the inverse fourth power of a passage width; and blocked passages, where two repulsions add up in front of a free gap.
- Critical points other than the goal cannot be avoided by any potential; navigation functions make them all saddles on sphere worlds, harmonic potentials need a global grid computation, random walks cost time and path quality. The remedy that scales is to use the field as a local layer under a global plan, with a stalled leg triggering replanning.
- Inter-agent repulsion is Reynolds's separation rule, cohesion a spring to the neighbours' centroid, alignment a velocity consensus. Potential fields survive in the swarm as soft spacing and formation keeping, while ORCA and DWA take the safety layer.

Further reading. Khatib [6] introduced the artificial potential field with the FIRAS repulsion; Koren and Borenstein [8] is the critique of Section 1.7.5, still worth reading for its experiments, and Borenstein and Koren [2] is the vector field histogram they proposed instead; Ge and Cui [5] named GNRRON and proposed the d^n correction. Koditschek and Rimon [7] and Rimon and Koditschek [12] define navigation functions, prove the count of saddles and construct Equation (1.12); Connolly, Burns and Weiss [4] introduced harmonic potentials and Barraquand and Latombe [1] the random-walk escape. Reynolds [11] is the flocking paper and Olfati-Saber [10] its control-theoretic form; Choset et al. [3, ch. 4] and LaValle [9] give the textbook treatments.

1.13 Exercises

Exercise 1.1 (★★ Derivation). (a) Derive the repulsive force Equation (1.5) from the potential Equation (1.4) and show that the force is continuous at $\rho = \rho_0$. (b) Give ρ and $\nabla \rho$ for a disc and for a segment obstacle (??), and say where $\nabla \rho$ is discontinuous. (c) Derive the corrected force Equation (1.9) from Equation (1.8) and show that at the goal it vanishes for $n \geq 2$ but not for $n = 1$.

Exercise 1.2 (★). (a) Verify that the hybrid potential Equation (1.3) and its force are continuous at $d = d^*$. (b) Show that clipping the quadratic force at the speed v_{max} gives the same velocity command as the hybrid potential with $d^* = v_{\text{max}}/k_{\text{att}}$. (c) Without obstacles and without a speed limit, solve $\dot{\mathbf{p}} = -k_{\text{att}}(\mathbf{p} - \mathbf{p}_g)$ and show that the drone never reaches the goal exactly; how long does it take to get within ε_g from a distance D ?

Exercise 1.3 (★★). Take the scene of Example 1.5 and the axis $y = 4$. (a) Derive Equation (1.7), show that it has a root in $(1, 3)$, and compute the root by bisection. (b) Estimate the Hessian of U at the root by central differences of the analytic force and classify the critical point. Which starts are trapped? (c) Repeat (b) for the stopping point $(2.676, 4.1)$ of the basin experiment and explain why the second obstacle turns a saddle into a minimum.

Exercise 1.4 (★★). (a) Show that with the factor d^n the total potential is non-negative with its global minimum at the goal, and that for $n \geq 2$ the goal is a critical point. (b) Run the corrected field with $n = 1$ and a goal tolerance below the chatter amplitude $\Delta t v_{\text{max}}$, `simulate((8,6), (5.4,4), DISC, replace(prm, n_gnrron=1,`

`goal_tol=1e-3`)), describe the motion near the goal and explain it with [Exercise 1.1\(c\)](#); repeat with $n = 2$ and with the chapter's tolerance $\varepsilon_g = 0.05$ and say what each run reports. (c) Run the local-minimum case of [Section 1.7.1](#) with $n = 2$ to show that the correction does not remove local minima elsewhere, and explain why the stopping point moves.

Exercise 1.5 (★★). (a) Without obstacles and without a speed limit, write the update of [Algorithm 1.2](#) for the quadratic bowl as a linear recursion for the error $\mathbf{p}_k - \mathbf{p}_g$ and find the values of $k_{\text{att}}\Delta t$ for which it converges monotonically, with alternating sign, or diverges. (b) Generalise to an arbitrary minimum with Hessian $\mathbf{H}$ and derive the condition of [Proposition 1.10](#). (c) Derive [Equation \(1.10\)](#), evaluate it for the widths 0.6, 0.8 and 1.0, and explain which runs of [Table 1.3](#) the linear analysis does and does not describe.

Exercise 1.6 (★★). (a) Derive [Equation \(1.11\)](#) for the passage of [Section 1.7.4](#) and locate its root numerically. (b) With $k_{\text{rep}} = 1$ and $\rho_0 = 2$ fixed, find by experiment the smallest k_{att} for which the drone from $(0, 4.3)$ enters the passage of width 0.6. (c) Rerun [Example 1.5](#) with that k_{att} and report the minimum clearance from $(0, 4.5)$; what does the comparison say about tuning one pair of gains for a whole map? (d) Explain why a waypoint inside the passage removes the problem without touching the gains.

Exercise 1.7 (★). (a) Explain why the force $\mathbf{F}$ is continuous but not continuously differentiable across the influence boundary $\rho = \rho_0$ ([Proposition 1.6](#)), and what that costs a controller that estimates the stiffness of [Equation \(1.10\)](#) by finite differences of the force. (b) The stuck detector of line 8 fires when the drone has moved less than δ in the last W steps. Explain why it cannot distinguish a local minimum from a very flat region of U , give one scene of each kind, and argue why the caller of [Algorithm 1.2](#) should treat both in the same way.

Exercise 1.8 (★★★ Coding). Extend `simulate_swarm()` with Reynolds's alignment and cohesion rules, the velocity update [Equation \(1.14\)](#) over the neighbours within ρ_a and a spring towards their centroid. Release twenty drones with random positions and velocities and plot the spread of the velocities and of the positions against time. Show that the velocities converge to a common value when the neighbourhood graph stays connected, construct an initial configuration in which the swarm splits into two flocks that never meet, and relate both outcomes to the consensus results of ??.

Exercise 1.9 (★★★ Coding). This is the Week 7 coding exercise of the study plan (??): implement a potential-field controller in a continuous 2D environment and create its failure cases. Do not read `ch15_potential_fields.py` until you are stuck. (a) Implement [Algorithms 1.1](#) and [1.2](#) for disc and segment obstacles and check your forces against central differences of your potential. (b) Reproduce the four failure modes of [Section 1.7](#) in scenes of your own – a local minimum, a goal inside ρ_0 (then fix it with $n = 2$), an oscillating and a crashing run in a passage, a free passage that is never entered – each with the figure and the number that explains it (the root of F_x , the offset d , the stability number $\Delta t \lambda_{\text{max}}$). (c) Sweep a grid of starts as in [Section 1.8.1](#) for a scene with a U-shaped obstacle and measure the fraction that arrives, with and without random-walk escapes. (d) Plan a path on a grid with A^* (??), inflate the obstacles, use your controller as the local layer along the waypoints, compare the arrival fraction with (c), and report what happens when a waypoint lies within ρ_0 of a wall.

Part V

Sampling-Based Motion Planning

Part VI

Tracking and Prediction

Part VII

Optimization and Control

Part VIII

Putting It Together

Part IX

Appendices

Bibliography

- [1] J. Barraquand and J.-C. Latombe. “Robot Motion Planning: A Distributed Representation Approach”. In: *The International Journal of Robotics Research* 10.6 (1991), pp. 628–649.
- [2] J. Borenstein and Y. Koren. “The Vector Field Histogram—Fast Obstacle Avoidance for Mobile Robots”. In: *IEEE Transactions on Robotics and Automation* 7.3 (1991), pp. 278–288.
- [3] H. Choset, K. M. Lynch, S. Hutchinson, G. Kantor, W. Burgard, L. E. Kavraki, and S. Thrun. *Principles of Robot Motion: Theory, Algorithms, and Implementations*. MIT Press, 2005.
- [4] C. I. Connolly, J. B. Burns, and R. Weiss. “Path Planning Using Laplace’s Equation”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 1990, pp. 2102–2106.
- [5] S. S. Ge and Y. J. Cui. “New Potential Functions for Mobile Robot Path Planning”. In: *IEEE Transactions on Robotics and Automation* 16.5 (2000), pp. 615–620.
- [6] O. Khatib. “Real-Time Obstacle Avoidance for Manipulators and Mobile Robots”. In: *The International Journal of Robotics Research* 5.1 (1986), pp. 90–98.
- [7] D. E. Koditschek and E. Rimon. “Robot Navigation Functions on Manifolds with Boundary”. In: *Advances in Applied Mathematics* 11.4 (1990), pp. 412–442.
- [8] Y. Koren and J. Borenstein. “Potential Field Methods and Their Inherent Limitations for Mobile Robot Navigation”. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 1991, pp. 1398–1404.
- [9] S. M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006.
- [10] R. Olfati-Saber. “Flocking for Multi-Agent Dynamic Systems: Algorithms and Theory”. In: *IEEE Transactions on Automatic Control* 51.3 (2006), pp. 401–420.
- [11] C. W. Reynolds. “Flocks, Herds and Schools: A Distributed Behavioral Model”. In: *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*. 1987, pp. 25–34.
- [12] E. Rimon and D. E. Koditschek. “Exact Robot Navigation Using Artificial Potential Functions”. In: *IEEE Transactions on Robotics and Automation* 8.5 (1992), pp. 501–518.

Index

- artificial potential field, 5
 - attractive potential, 7
 - influence distance, 7
 - local minimum, 8, 14
 - local-minimum detection, 9
 - repulsive potential, 7
- basin of a local minimum, 8
- clearance, 7
- critical point, 8
- dynamic window approach, 5
- flocking
 - alignment, 21
 - cohesion, 21
 - separation, 21
- force (potential field), 6, 7
- GNRON (goals non-reachable with obstacles nearby), 14
- harmonic potential, 19
- inter-agent repulsion, 20
- Koren–Borenstein critique, 18
- navigation function, 18
- potential, 6, 7
- random walk (escape from local minima), 19
- saddle point, 14
- simulated annealing, 19
- sphere world, 18
- stiffness of a potential, 16
- vector field histogram, 17
- velocity limit, 9